

DiffuChannel benchmark: diffusion vs DeepGANnel across all phenotypes

Barrett-Jolley lab

01 September 2026

Contents

Purpose	1
Configuration	1
Reading records	3
Metric functions	6
1. Find the files	12
2. Pre-flight	14
Sample rate	16
Noise level and segment length for the spectral metrics	18
3. Per-phenotype analysis	21
Equivalent versus honest	26
4. Scoring	28
Head to head, per phenotype	31
5. Pooled comparison	32
What this many phenotypes will and will not support	35
6. Publication figures	36
Figure: example traces	36
Figure: idealisations	37
Figure: all-points amplitude histograms	38
Figure: single-level noise	39
Figure: dwell times	41
Bursts	44
Burst Figures	47
Label self-consistency	51
Figure: slow envelope	58
Figure: manifold projections (optional)	59
7. Numbers for the manuscript	64
What this comparison does and does not settle	65
Session information	65

Purpose

One pass over every phenotype folder. For each one it reads the real seed record, the DeepGANnel record and the diffusion (DDPM) record, computes the same metric set on the two synthetic arms, and then pools the five paired comparisons.

Every arm is scored identically. Whether that flatters the diffusion model OR NOT is not this paper's business.

```
required <- c("dplyr", "tidyr", "ggplot2", "readr", "purrr", "arrow",
             "scales", "patchwork", "tibble", "Rtsne", "uwot", "kableExtra")
missing <- required[!vapply(required, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing)) {
  stop("Install first: install.packages(c(\"",
    paste(missing, collapse = "\", \""), "\"))")
}

library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(purrr)
library(arrow)
library(patchwork)
library(uwot)
library(Rtsne)

set.seed(42)
```

Configuration

```
CFG <- list(
  # ---- where things live -----
  pheno_root = "./Phenotypes", # holds Phenotype_A, Phenotype_B, ...
  batch_dir  = "./batch_out",  # holds Phenotype_A_DDPM.parquet, ...
  out_dir    = "./benchmark_out",

  raw_pattern = "raw",          # substring marking the seed csv (case-insensitive)
  gan_pattern = "gan",          # substring marking the DeepGANnel csv
  only        = NULL,           # e.g. c("Phenotype_A") to run a subset

  # ---- recording parameters -----
  sample_hz = 1e4,              # fallback only; a Time column always wins
  filter_hz = 1e3,

  # ---- how to treat a generated file's own time column -----
  # "seed" every arm inherits the raw record's rate (treat as equivalent:
  #         the generated file's time column is cosmetic and wrong)
  # "own"  an arm with a time column is taken at its word (treat as honest:
  #         it really was produced at that rate)
  rate_policy = "seed",
  rate_compare = TRUE,          # also run the other policy where they differ

  # ---- must match training -----
```

```

seq_len      = 128,
slow_factor  = 64,

# ---- analysis knobs -----
n_windows_mmd = 400,          #  $O(n^2)$  in the kernel: keep modest
n_perm       = 200,
# Spectral segment length. "auto" is resolved per phenotype (see below); a
# number is applied to every phenotype and must fit all of them.
psd_seg      = "auto",
psd_seg_min  = 256,          # below this, stop rather than report a coarse PSD
psd_n_seg_min = 20,          # target segments per arm; fewer is flagged, not fatal
equalise_noise = TRUE,       # hold every arm to the same segment and sample count
acf_lag_max  = 200,
# Which conductance level supplies the noise segments. "auto" picks, per
# phenotype, whichever of closed or open every arm can support; a record that
# barely closes has no baseline but plenty of open-level material.
noise_level  = "auto",       # "auto", "closed" or "open"
ks_subsample = 2e5,
trace_secs   = 0.3,

# ---- optional extras -----
do_manifold  = TRUE,         # UMAP / t-SNE, needs uwot + Rtsne, slow
n_windows_proj = 400,

proj_win     = 512,

cache = FALSE                # reuse per-phenotype results if present
#Careful with caching - it can cause real confusion!!
)

dir.create(CFG$out_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(file.path(CFG$out_dir, "cache"), showWarnings = FALSE, recursive = TRUE)

pal <- c("Real" = "#1b1b1b", "Diffusion" = "#c0392b", "GAN" = "#8e44ad",
        "Real floor" = "#7f8c8d", "White noise" = "#bdc3c7")

theme_channel <- function(base_size = 10) {
  theme_classic(base_size = base_size) +
    theme(strip.background = element_blank(),
          strip.text = element_text(face = "bold", hjust = 0),
          legend.position = "top",
          legend.title = element_blank(),
          legend.key.size = unit(0.8, "lines"),
          axis.text = element_text(colour = "black"))
}

save_fig <- function(plot, name, width = 8, height = 5) {
  ggsave(file.path(CFG$out_dir, paste0(name, ".pdf")), plot,
          width = width, height = height)
  ggsave(file.path(CFG$out_dir, paste0(name, ".png")), plot,
          width = width, height = height, dpi = 300)
  invisible(plot)
}

```

```

# kable(), scaled to fit the page in PDF so wide tables stop cramming their
# headers together. latex_options do nothing when knitting to HTML, so this
# is safe either way. hold_position stops the table drifting away from its
# caption once it's wrapped in a resize box.
nice_kable <- function(x, caption = NULL, ...) {
  if (!is.null(caption) && knitr::is_latex_output())
    caption <- gsub("([_&%#])", "\\\\\\\\1", caption)
  knitr::kable(x, caption = caption, ...) |>
    kableExtra::kable_styling(latex_options = c("scale_down", "hold_position"))
}

```

Reading records

The seed and GAN csv files were not written by this pipeline, so their headers cannot be assumed. Named columns are used when they are recognisable; otherwise the columns are inferred the same way `batch_run.py` infers them, which is the only way the R and Python sides can agree on which column was which. The resolved mapping is printed in the manifest below and is worth reading once.

```

normalise_names <- function(nms) gsub("[_]", " ", trimws(tolower(nms)))

# 10%-of-range quantisation, as in batch_run.py stage_seed()
quantise <- function(x) {
  span <- diff(range(x, na.rm = TRUE))
  if (span > 0) round(x / (0.1 * span)) else rep(0, length(x))
}

# A channels column may hold no more than this many discrete levels (MAX_LEVELS
# in batch_run.py). Used both to infer the column and to check a named one.
MAX_LEVELS <- 10

is_time_col <- function(x) {
  d <- diff(x)
  length(d) > 1 && all(d > 0) &&
    isTRUE(all.equal(d, rep(d[1], length(d)), tolerance = 1e-3))
}

infer_cols <- function(df) {
  # The column count decides whether a time column exists at all: three columns
  # means one of them is time, two means there is none. Which one it is still
  # has to be found, because the order varies.
  time_col <- NA_integer_
  if (ncol(df) >= 3) {
    hit <- which(vapply(df, is_time_col, logical(1)))
    if (!length(hit)) {
      stop(ncol(df), " columns but none of them increases in constant steps, ",
           "so the time column cannot be identified")
    }
    time_col <- hit[1]
  }
  rest <- setdiff(seq_along(df), if (is.na(time_col)) integer(0) else time_col)
  lev <- lapply(rest, function(j) sort(unique(quantise(df[[j]]))))
  n <- vapply(lev, length, integer(1))
  k <- which.min(n)
}

```

```

if (n[k] > MAX_LEVELS) {
  stop("no column with <= ", MAX_LEVELS, " discrete levels; level counts ",
       paste(n, collapse = ", "))
}
others <- setdiff(seq_along(rest), k)
list(time = time_col,
     chan = rest[k],
     cur = rest[others[which.max(n[others])]],
     levels = lev[[k]])
}

read_record <- function(path) {
  df <- if (grepl("\\.parquet$", path, ignore.case = TRUE)) {
    as.data.frame(arrow::read_parquet(path))
  } else {
    d <- suppressWarnings(as.data.frame(
      readr::read_csv(path, show_col_types = FALSE, name_repair = "minimal")))
    # A file with no header gets its first data row promoted to names; spot that
    # by the names parsing as numbers, and re-read.
    if (all(!is.na(suppressWarnings(as.numeric(names(d)))))) {
      d <- as.data.frame(readr::read_csv(path, col_names = FALSE,
                                         show_col_types = FALSE))
    }
    d
  }

  df <- df[, vapply(df, function(x) is.numeric(x) || is.logical(x) ||
                    !all(is.na(suppressWarnings(as.numeric(as.character(x))))),
                    logical(1)), drop = FALSE]
  df[] <- lapply(df, function(x) suppressWarnings(as.numeric(as.character(x))))
  df <- df[stats::complete.cases(df), , drop = FALSE]
  if (ncol(df) < 2 || nrow(df) < 100) stop("too few usable rows/columns")

  nn <- normalise_names(names(df))
  idx <- function(target) { h <- which(nn %in% target); if (length(h)) h[1] else NA_integer_ }
  i_chan <- idx(c("channels", "channel", "ideal", "idealisation", "idealization"))
  i_cur <- idx(c("noisy current", "current", "raw", "raw current", "i"))
  i_time <- idx(c("time", "t"))

  # Headers are either absent or correct, so a recognised pair is taken at face
# value. Anything else is inferred. Column order is never assumed either way.
  if (!is.na(i_chan) && !is.na(i_cur) && i_chan != i_cur) {
    how <- "named columns"
    chan <- as.integer(round(df[[i_chan]]))
  } else {
    inf <- infer_cols(df)
    i_chan <- inf$chan; i_cur <- inf$cur; i_time <- inf$time
    how <- "inferred"
    chan <- match(quantise(df[[i_chan]]), inf$levels) - 1L
  }

  out <- tibble::tibble(channels = as.numeric(chan),
                       current = as.numeric(df[[i_cur]]))

```

```

out$time <- if (!is.na(i_time)) as.numeric(df[[i_time]]) else
  (seq_len(nrow(out)) - 1) / CFG$sample_hz
out <- dplyr::filter(out, is.finite(channels), is.finite(current))

attr(out, "map") <- sprintf(
  "%s: chan=col%d cur=col%d time=%s", how, i_chan, i_cur,
  if (is.na(i_time)) sprintf("none, assuming %g Hz", CFG$sample_hz)
  else paste0("col", i_time))
dplyr::select(out, time, channels, current)
}

fs_of <- function(d, fallback = CFG$sample_hz) {
  if (nrow(d) < 3) return(fallback)
  dt <- stats::median(diff(d$time), na.rm = TRUE)
  if (!is.finite(dt) || dt <= 0) fallback else 1 / dt
}

#' Crop a record to its first n rows, keeping the column map
#'
#' Subsetting a tibble drops attributes, so the "map" attribute that
#' read_record attached is stashed and restored. Anything reading it
#' downstream (inventory's has_time, the manifest print) would otherwise
#' see NULL.
#'
#' @param d A record as returned by read_record: a tibble with columns
#'         time, channels, current, and a "map" attribute describing
#'         which source column supplied each.
#' @param n Integer, the number of rows to keep, counted from the start
#'         of the record. Assumed to be no greater than nrow(d).
#' @return The same tibble with rows 1:n and the "map" attribute intact.
crop_rows <- function(d, n) {
  m <- attr(d, "map")
  d <- d[seq_len(n), ]
  attr(d, "map") <- m
  d
}

```

Metric functions

Carried over from DiffuChannel_analysis.Rmd without change.

```

make_windows <- function(x, win, n_max = Inf, stride = NULL) {
  n <- length(x)
  if (n < win) return(matrix(numeric(0), nrow = 0, ncol = win))
  if (is.null(stride)) stride <- max(1L, floor((n - win) / max(1L, min(n_max, n))))
  starts <- seq(1L, n - win + 1L, by = stride)
  if (length(starts) > n_max) starts <- sample(starts, n_max)
  t(vapply(starts, function(s) x[s:(s + win - 1L)], numeric(win)))
}

sq_dists <- function(A, B) {
  pmax(outer(rowSums(A^2), rowSums(B^2), "+") - 2 * tcrossprod(A, B), 0)
}

```

```

# Median heuristic: gamma = 1 / median pairwise squared distance. The fixed 1/L
# used during training underflows to zero on current in pA and pins the
# statistic to its finite-sample floor.
median_gamma <- function(X, Y, n_sub = 200) {
  A <- X[sample(nrow(X), min(n_sub, nrow(X))), , drop = FALSE]
  B <- Y[sample(nrow(Y), min(n_sub, nrow(Y))), , drop = FALSE]
  m <- stats::median(sq_dists(A, B))
  if (!is.finite(m) || m <= 0) 1 / ncol(X) else 1 / m
}

mmd_rbf <- function(X, Y, gamma = NULL, unbiased = TRUE) {
  m <- nrow(X); n <- nrow(Y)
  if (m < 2 || n < 2) return(NA_real_)
  if (is.null(gamma)) gamma <- median_gamma(X, Y)
  Kxx <- exp(-gamma * sq_dists(X, X))
  Kyy <- exp(-gamma * sq_dists(Y, Y))
  Kxy <- exp(-gamma * sq_dists(X, Y))
  if (unbiased) {
    (sum(Kxx) - sum(diag(Kxx))) / (m * (m - 1)) +
    (sum(Kyy) - sum(diag(Kyy))) / (n * (n - 1)) - 2 * mean(Kxy)
  } else {
    mean(Kxx) + mean(Kyy) - 2 * mean(Kxy)
  }
}

mmd_permutation_p <- function(X, Y, n_perm = CFG$n_perm, gamma = NULL) {
  if (is.null(gamma)) gamma <- median_gamma(X, Y)
  obs <- mmd_rbf(X, Y, gamma)
  Z <- rbind(X, Y); m <- nrow(X)
  null <- replicate(n_perm, {
    i <- sample(nrow(Z))
    mmd_rbf(Z[i[seq_len(m)], , drop = FALSE], Z[i[-seq_len(m)], , drop = FALSE], gamma)
  })
  s <- stats::sd(null)
  list(mmd = obs,
       p = (1 + sum(null >= obs)) / (n_perm + 1),
       z = if (is.finite(s) && s > 0) (obs - mean(null)) / s else NA_real_)
}

# Noise segments are taken at a single conductance level, so that no gating step
# falls inside a segment. Runs are returned separately and never concatenated:
# gluing them together puts a step discontinuity at every join, and a step is
# broadband, lifting the high-frequency end of the spectrum and dragging the ACF
# towards zero in proportion to how often the record gates. That bias falls
# hardest on exactly the busy records the comparison is meant to include.
at_level <- function(d, level) {
  if (identical(level, "closed")) d$channels == 0 else d$channels >= max(d$channels)
}

level_runs <- function(d, level, min_len = 1L) {
  r <- rle(at_level(d, level))
  ends <- cumsum(r$lengths); starts <- ends - r$lengths + 1
  keep <- which(r$values & r$lengths >= min_len)
}

```

```

  lapply(keep, function(i) d$current[starts[i]:ends[i]])
}

longest_at <- function(d, level) {
  r <- rle(at_level(d, level))
  if (!any(r$values)) 0L else max(r$lengths[r$values])
}

# Lengths of the runs at one level, without materialising the runs themselves.
# Used to choose a segment length before any spectrum is computed.
run_lengths_at <- function(d, level) {
  r <- rle(at_level(d, level))
  r$lengths[r$values]
}

# How many Welch segments a set of run lengths yields at 50% overlap. Mirrors
# the loop in welch_psd, so the two cannot drift apart.
seg_count <- function(lens, nperseg) {
  lens <- lens[lens >= nperseg]
  if (!length(lens)) return(0L)
  as.integer(sum(1L + (lens - nperseg) %/% (nperseg %/% 2L)))
}

# Welch across a list of runs: every segment from every run long enough to hold
# one contributes to a single average periodogram.
#
# n_seg_max thins that set to a fixed count, taken evenly across the record. A
# Welch estimate's variance falls as 1/n_seg, so an arm that gates more often
# has shorter runs, fewer segments and a noisier spectrum, and the RMS distance
# to real inflates for reasons that have nothing to do with the model. Holding
# both synthetic arms to the same count makes that variance common to both and
# cancels it from their difference.
welch_psd <- function(runs, fs, nperseg, n_seg_max = Inf) {
  if (is.numeric(runs)) runs <- list(runs)
  runs <- runs[vapply(runs, length, integer(1)) >= nperseg]
  if (!length(runs)) stop("no run at the chosen level holds ", nperseg, " samples")

  step <- nperseg %/% 2
  idx <- do.call(rbind, lapply(seq_along(runs), function(i)
    cbind(run = i, start = seq(1L, length(runs[[i]]) - nperseg + 1L, by = step))))
  n_avail <- nrow(idx)

  if (is.finite(n_seg_max) && n_seg_max < n_avail) {
    k <- as.integer(n_seg_max)
    idx <- idx[1L + ((0:(k - 1L)) * n_avail) %/% k, , drop = FALSE]
  }

  w <- 0.5 - 0.5 * cos(2 * pi * (seq_len(nperseg) - 1) / (nperseg - 1)) # Hann
  U <- sum(w^2)
  acc <- numeric(nperseg %/% 2 + 1)
  for (r in seq_len(nrow(idx))) {
    s <- idx[r, "start"]
    seg <- runs[[idx[r, "run"]]][s:(s + nperseg - 1L)]

```



```

    seg <- seg - mean(seg)
    X <- stats::fft(seg * w)[seq_len(nperseg %/% 2 + 1)]
    acc <- acc + (Mod(X)^2) / (fs * U)
  }
  nseg <- nrow(idx)
  psd <- acc / nseg
  psd[-c(1, length(psd))] <- 2 * psd[-c(1, length(psd))]
  tibble::tibble(freq = seq(0, fs / 2, length.out = length(psd)), psd = psd,
                 n_seg = nseg, n_seg_avail = n_avail) |>
    dplyr::filter(freq > 0)
}

# Length-weighted mean of the per-run ACFs, for the same reason. n_max is the
# same argument as n_seg_max above: the ACF's sampling variance also falls with
# the material behind it, so arms are held to a common sample budget, spent on
# the runs in order and trimming the one that crosses it.
acf_of <- function(runs, lag_max, fs, n_max = Inf) {
  if (is.numeric(runs)) runs <- list(runs)
  runs <- runs[vapply(runs, length, integer(1)) > lag_max]
  if (!length(runs)) stop("no run at the chosen level is longer than ", lag_max,
                        " samples")

  if (is.finite(n_max)) {
    lens <- vapply(runs, length, integer(1))
    spent <- c(0L, cumsum(lens))[seq_along(lens)]
    take <- pmin(lens, pmax(0L, as.integer(n_max) - spent))
    keep <- take > lag_max
    if (any(keep)) runs <- Map(function(x, k) x[seq_len(k)], runs[keep], take[keep])
  }

  w <- vapply(runs, length, integer(1))
  A <- vapply(runs, function(x)
    stats::acf(x - mean(x), lag.max = lag_max, plot = FALSE,
               demean = TRUE)$acf[, 1, 1], numeric(lag_max + 1))
  tibble::tibble(lag_n = seq_len(lag_max + 1),
                 lag_ms = 1000 * (seq_len(lag_max + 1) - 1) / fs,
                 acf = as.numeric(A %*% w / sum(w)),
                 n_used = sum(w))
}

dwell_times <- function(ideal, fs) {
  r <- rle(as.integer(round(ideal)))
  tibble::tibble(level = r$values, n_sample = r$lengths,
                 dwell_ms = 1000 * r$lengths / fs) |>
    dplyr::mutate(state = ifelse(level == 0, "Closed", "Open"))
}

# Half-amplitude idealiser. Two Gaussians of common width, fitted by EM.
# Hard assignment fails when one level holds 1% of the record: the far tail of
# the crowded level outnumbers the sparse one on its own side of any threshold
# and drags the estimate onto itself. Responsibilities do not have that
# problem. Polarity says which of the two fitted levels is shut.
idealise_half_amplitude <- function(current, unit_pA = NULL, baseline = NULL,

```

```

                                n_levels = 1, polarity = 1) {
x <- current[is.finite(current)]

if (is.null(baseline) || is.null(unit_pA)) {
  # Fitting on a regular subsample keeps the knit time sane; a level holding
  # 1% of the record still contributes ~1000 points here.
  xf <- if (length(x) > 1e5) x[seq(1, length(x), length.out = 1e5)] else x

  # Seed outside both bulks, not at the 1st/99th percentile: in a record
  # that is open 99% of the time the 1st percentile is still open. The
  # starting weight is taken from the split at the midpoint rather than set
  # to a half, so the sparse component is not invited to bid for the
  # crowded one.
  mu <- stats::quantile(xf, c(0.001, 0.999), names = FALSE)
  sg <- max(stats::mad(xf), .Machine$double.eps^0.5)
  w <- min(max(mean(xf < mean(mu)), 1e-3), 1 - 1e-3)
  tol <- 1e-8 * sg

  for (i in 1:300) {
    d1 <- stats::dnorm(xf, mu[1], sg) * w
    d2 <- stats::dnorm(xf, mu[2], sg) * (1 - w)
    r <- d2 / (d1 + d2)
    r[!is.finite(r)] <- 0.5
    s2 <- sum(r); s1 <- sum(1 - r)
    if (s1 < 1 || s2 < 1) break
    new_mu <- c(sum((1 - r) * xf) / s1, sum(r * xf) / s2)
    new_sg <- max(sqrt((sum((1 - r) * (xf - new_mu[1])^2) +
                        sum(r * (xf - new_mu[2])^2)) / length(xf)),
                  .Machine$double.eps^0.5)
    w <- s1 / length(xf)
    done <- max(abs(c(new_mu - mu, new_sg - sg))) < tol
    mu <- new_mu; sg <- new_sg
    if (done) break
  }

  mu <- sort(mu)
  shut <- if (polarity >= 0) mu[1] else mu[2]
  open <- if (polarity >= 0) mu[2] else mu[1]
  minority <- min(w, 1 - w)

  if (!is.finite(open - shut) || abs(open - shut) < 3 * sg ||
      minority < 5e-4) {
    out <- rep(NA_integer_, length(current))
    attr(out, "why") <- sprintf(
      "sep %.3g, width %.3g, rarer level %.3g",
      abs(open - shut), sg, minority)
    return(out)
  }

  if (is.null(baseline)) baseline <- shut
  if (is.null(unit_pA)) unit_pA <- open - shut
}

```

```

lv1 <- (current - baseline) / unit_pA
pmin(pmax(as.integer(round(lv1)), 0L), as.integer(n_levels))
}

idealise_label_anchored_cv <- function(
  current,
  labels,
  n_levels = 1L,
  polarity = 1,
  block_len = 1000L,
  trim = 0.10,
  min_state_n = 50L
) {
  stopifnot(length(current) == length(labels))

  ok <- is.finite(current) & is.finite(labels)
  labels <- as.integer(labels > 0)

  n <- length(current)
  block <- ((seq_len(n) - 1L) %/% block_len) + 1L
  fold <- block %/% 2L

  recovered <- rep(NA_integer_, n)
  fit_info <- vector("list", 2L)

  robust_centre <- function(x) {
    x <- x[is.finite(x)]

    if (!length(x)) {
      return(NA_real_)
    }

    mean(x, trim = trim)
  }

  for (test_fold in 0:1) {
    train <- ok & fold != test_fold
    test <- ok & fold == test_fold

    x_closed <- current[train & labels == 0L]
    x_open <- current[train & labels > 0L]

    if (length(x_closed) < min_state_n ||
        length(x_open) < min_state_n) {
      next
    }

    closed_centre <- robust_centre(x_closed)
    open_centre <- robust_centre(x_open)

    if (!is.finite(closed_centre) ||
        !is.finite(open_centre) ||
        closed_centre == open_centre) {

```

```

    next
  }

  # Respect the polarity defined from the real record.
  if (polarity >= 0) {
    baseline <- closed_centre
    unit_pA <- open_centre - closed_centre
  } else {
    baseline <- closed_centre
    unit_pA <- open_centre - closed_centre
  }

  lvl <- (current[test] - baseline) / unit_pA

  recovered[test] <- pmin(
    pmax(as.integer(round(lvl)), 0L),
    as.integer(n_levels)
  )

  fit_info[[test_fold + 1L]] <- list(
    n_closed = length(x_closed),
    n_open = length(x_open),
    baseline = baseline,
    open_centre = open_centre,
    unit_pA = unit_pA
  )
}

if (all(is.na(recovered))) {
  attr(recovered, "why") <-
    "cross-fitted label anchoring lacked enough observations in one or both states"
}

attr(recovered, "fit") <- fit_info
attr(recovered, "method") <- "cross-fitted label-anchored half-amplitude"

recovered
}

# Cohen's kappa on two logical vectors. Chance agreement is high when a record
# is nearly always open or nearly always shut, which is exactly when raw
# agreement flatters an arm, so kappa rather than accuracy.
cohen_kappa <- function(a, b) {
  po <- mean(a == b)
  pe <- mean(a) * mean(b) + mean(!a) * mean(!b)
  if (!is.finite(pe) || pe >= 1) NA_real_ else (po - pe) / (1 - pe)
}

block_mean <- function(x, factor) {
  factor <- max(1L, as.integer(factor))
  nb <- length(x) %/% factor
  if (nb < 1) return(mean(x))
  colMeans(matrix(x[seq_len(nb * factor)], nrow = factor))
}

```

```

ks_safe <- function(a, b, n_max = CFG$ks_subsample) {
  a <- a[is.finite(a)]; b <- b[is.finite(b)]
  if (length(a) > n_max) a <- sample(a, n_max)
  if (length(b) > n_max) b <- sample(b, n_max)
  suppressWarnings(stats::ks.test(a, b))
}

# Window choice for the example traces, ported from _best_window in
# batch_run.py so the R figures and the Python previews pick the same way:
# prefer a stretch whose Popen sits in [lo, hi], else the one nearest the middle
# of that band. Taking the first n samples instead lands on whatever the record
# happens to open with, which for a sparse phenotype is often no events at all.
best_window <- function(chan, n, lo = 0.4, hi = 0.8) {
  if (length(chan) <= n) return(1L)
  cs <- c(0, cumsum(as.numeric(chan)))
  k <- (cs[(n + 1):length(cs)] - cs[1:(length(cs) - n)]) / n
  inside <- which(k >= lo & k <= hi)
  if (length(inside)) return(inside[ceiling(length(inside) / 2)])
  which.min(abs(k - 0.5 * (lo + hi)))
}

rms_diff <- function(a, b) sqrt(mean((a - b)^2))

```

1. Find the files

```

find_one <- function(folder, pattern) {
  f <- list.files(
    folder,
    pattern = "\\.(csv|parquet)$",
    full.names = TRUE,
    ignore.case = TRUE
  )

  hit <- f[
    grepl(pattern, basename(f), ignore.case = TRUE)
  ]

  if (!length(hit)) return(NA_character_)

  if (length(hit) > 1) {
    warning(
      folder, ": ", length(hit),
      " files match '", pattern,
      "', using ", basename(hit[1])
    )
  }

  hit[1]
}

folders <- list.dirs(CFG$pheno_root, recursive = FALSE)
if (!is.null(CFG$only)) folders <- folders[basename(folders) %in% CFG$only]

```

```

if (!length(folders)) stop("No phenotype folders in ", CFG$pheno_root)

parquets <- list.files(CFG$batch_dir, pattern = "\\..parquet$", full.names = TRUE)

manifest <- purrr::map_dfr(folders, function(fd) {
  tag <- gsub(" ", "_", basename(fd))
  ddpm <- parquets[grepl(paste0("^", tag), basename(parquets))]
  if (!length(ddpm)) ddpm <- parquets[grepl(tag, basename(parquets), fixed = TRUE)]
  tibble::tibble(
    phenotype = tag,
    raw = find_one(fd, CFG$raw_pattern),
    gan = find_one(fd, CFG$gan_pattern),
    ddpm = if (length(ddpm)) ddpm[1] else NA_character_
  )
})

nice_kable(
  manifest |> dplyr::mutate(dplyr::across(c(raw, gan, ddpm), basename)),
  caption = "Manifest. Check that each row's three files belong to the same phenotype."
)

```

Table 1: Manifest. Check that each row's three files belong to the same phenotype.

phenotype	raw	gan	ddpm
Phenotype_A	4096PYRraw.csv	4096PYRGAN.csv	Phenotype_A_DDPM.parquet
Phenotype_B	4096edr01raw.csv	4096edr01GAN.csv	Phenotype_B_DDPM.parquet
Phenotype_C	4096lina11raw.csv	4096Lina2GAN_data.csv	Phenotype_C_DDPM.parquet
Phenotype_D	pvnraw.csv	pvn_GAN_512butt.parquet	Phenotype_D_DDPM.parquet
Phenotype_E	c035[NF=50Hz]0-32_4kraw.csv	c035[NF=50Hz]0-32_4kGAN.csv	Phenotype_E_DDPM.parquet

```

bad <- manifest |> dplyr::filter(is.na(raw) | is.na(ddpm))
if (nrow(bad)) {
  warning("Dropping phenotypes with no seed or no DDPM file: ",
    paste(bad$phenotype, collapse = ", "))
  manifest <- dplyr::filter(manifest, !is.na(raw), !is.na(ddpm))
}
if (!nrow(manifest)) stop("Nothing left to analyse.")

```

2. Pre-flight

Every record is read and checked before a single metric is computed, so that a record which cannot support the analysis stops the run here rather than becoming an NA that propagates quietly into the pooled tables.

All records are held in memory for the rest of the notebook. That is the price of reading each file exactly once; if it becomes a problem, run fewer phenotypes at a time with `CFG$only`.

```

recs <- purrr::map(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  out <- list(Real = read_record(row$raw), Diffusion = read_record(row$ddpm))
  if (!is.na(row$gan)) out$GAN <- read_record(row$gan)
  out
})

```

```

})
names(recs) <- manifest$phenotype

n_before <- purrr::imap_dfr(recs, function(arms, ph)
  tibble::tibble(phenotype = ph, arm = names(arms),
    n_read = vapply(arms, nrow, integer(1))))

# Every arm of a phenotype is cropped to the row count of its shortest arm.
# Rows, not seconds and not file size: the metrics are computed per sample, and
# several of them carry a length bias. MMD's z-score scales with the number of
# windows, the permutation p loses power when there are fewer, the KS statistic
# has a finite-sample floor, and both spectral distances are differences between
# estimates whose variance falls with the material behind them. An arm that is
# merely shorter would otherwise score differently for that reason alone.
for (ph in names(recs)) {
  arms <- recs[[ph]]
  min_len <- Inf
  for (a in names(arms)) { min_len <- min(min_len, nrow(arms[[a]])) }
  for (a in names(arms)) { arms[[a]] <- crop_rows(arms[[a]], min_len) }
  recs[[ph]] <- arms
}

crop_report <- n_before |>
  dplyr::left_join(
    purrr::imap_dfr(recs, function(arms, ph)
      tibble::tibble(phenotype = ph, arm = names(arms),
        n_used = vapply(arms, nrow, integer(1)))),
    by = c("phenotype", "arm")) |>
  dplyr::mutate(kept = n_used / n_read)

nice_kable(crop_report, digits = 3,
  caption = "Rows read and rows retained after cropping each phenotype's arms to its shortest")

```

Table 2: Rows read and rows retained after cropping each phenotype's arms to its shortest.

phenotype	arm	n_read	n_used	kept
Phenotype_A	Real	73316	73316	1.000
Phenotype_A	Diffusion	100000	73316	0.733
Phenotype_A	GAN	97400	73316	0.753
Phenotype_B	Real	1000445	100000	0.100
Phenotype_B	Diffusion	100000	100000	1.000
Phenotype_B	GAN	490896	100000	0.204
Phenotype_C	Real	150752	100000	0.663
Phenotype_C	Diffusion	100000	100000	1.000
Phenotype_C	GAN	779200	100000	0.128
Phenotype_D	Real	255840	100000	0.391
Phenotype_D	Diffusion	100000	100000	1.000
Phenotype_D	GAN	1000000	100000	0.100
Phenotype_E	Real	366588	100000	0.273
Phenotype_E	Diffusion	100000	100000	1.000

Phenotype_E	GAN	278528	100000	0.359
-------------	-----	--------	--------	-------

```
readr::write_csv(crop_report, file.path(CFG$out_dir, "crop_report.csv"))

inventory <- purrr::imap_dfr(recs, function(arms, ph) {
  purrr::imap_dfr(arms, function(d, a) {
    closed <- d$current[d$channels == 0]
    open   <- d$current[d$channels > 0]
    tibble::tibble(
      phenotype = ph, arm = a, n = nrow(d),
      has_time = !grepl("time=none", attr(d, "map")),
      fs = fs_of(d),
      longest_closed = longest_at(d, "closed"),
      longest_open   = longest_at(d, "open"),
      popen = mean(d$channels > 0), max_level = max(d$channels),
      baseline = median(closed),
      unit_amp = median(open) - median(closed),
      cols = attr(d, "map"))
  })
})

# A record that has been min-max scaled, or left in the wrong units, has a
# unitary amplitude unlike its seed's. Every amplitude metric then compares
# units rather than distributions, and the arm loses on bookkeeping.
scale_check <- inventory |>
  dplyr::group_by(phenotype) |>
  dplyr::mutate(rel = unit_amp / unit_amp[arm == "Real"]) |>
  dplyr::ungroup() |>
  dplyr::filter(arm != "Real", rel < 1 / 3 | rel > 3)
if (nrow(scale_check)) {
  stop("Unitary amplitude differs from the seed by more than 3x in: ",
    paste(sprintf("%s/%s (%.3g vs %.3g)", scale_check$phenotype,
      scale_check$arm, scale_check$unit_amp,
      scale_check$unit_amp / scale_check$rel), collapse = "; "),
    ". That is a scaling or units problem, not a modelling result: the ",
    "amplitude metrics would compare units rather than distributions. ",
    "Check whether a min-max scaled copy has been used.")
}

nice_kable(inventory, digits = 4,
  caption = "Pre-flight inventory. Every record that will be analysed.")
```

Table 3: Pre-flight inventory. Every record that will be

phenotype	arm	n	has_time	fs	longest_closed	longest_open	popen	max_level	baseline
Phenotype_A	Real	73316	FALSE	10000	3561	636	0.5825	1	0.1817
Phenotype_A	Diffusion	73316	TRUE	10000	1533	582	0.5784	1	0.2839
Phenotype_A	GAN	73316	FALSE	10000	2459	426	0.5461	1	0.0105
Phenotype_B	Real	100000	FALSE	10000	8490	698	0.0715	1	0.0000
Phenotype_B	Diffusion	100000	TRUE	10000	9668	520	0.0790	1	0.0404
Phenotype_B	GAN	100000	FALSE	10000	4668	667	0.1592	1	-0.1749
Phenotype_C	Real	100000	FALSE	10000	7709	5715	0.6037	1	-0.4164

Phenotype_C	Diffusion	100000	TRUE	10000	6570	3540	0.7548	1	-0.4215
Phenotype_C	GAN	100000	FALSE	10000	3597	1855	0.4858	1	-0.3740
Phenotype_D	Real	100000	TRUE	10000	10	1832	0.9883	1	0.3370
Phenotype_D	Diffusion	100000	TRUE	10000	17	1937	0.9879	1	0.3615
Phenotype_D	GAN	100000	TRUE	10000	6	950	0.9849	1	0.3299
Phenotype_E	Real	100000	TRUE	20000	4088	665	0.8171	1	3.9961
Phenotype_E	Diffusion	100000	TRUE	10000	284	616	0.8571	1	4.9219
Phenotype_E	GAN	100000	FALSE	10000	149	579	0.8804	1	5.0013

```
readr::write_csv(inventory, file.path(CFG$out_dir, "preflight_inventory.csv"))
```

Sample rate

Only the seed record's time column reflects a real acquisition. `diffusion_app.py` writes its export time as `arange(n) * d * SAMPLE_DT` with `SAMPLE_DT` fixed at `1e-4`, so a DDPM record always claims 10 kHz whatever it was trained on. A GAN record's time column is whatever DeepGANnel wrote, and when that disagrees with the seed there are two readings, neither of which can be settled from the file alone:

- **equivalent** (`rate_policy = "seed"`): the time column is cosmetic. The record is a sample-for-sample stand-in for the seed and inherits its rate.
- **honest** (`rate_policy = "own"`): the record really was produced at that rate, so it has half the bandwidth and its events last twice as long in wall-clock terms.

Relabelling a genuinely decimated record makes its dwell times and spectrum wrong; taking a cosmetic label at face value does the same in the other direction. So both are computed and compared below.

```
seed_fs <- inventory |>
  dplyr::filter(arm == "Real") |>
  dplyr::mutate(fs = dplyr::if_else(has_time, fs, CFG$sample_hz))
seed_fs <- setNames(seed_fs$fs, seed_fs$phenotype)

rates_under <- function(policy) {
  inventory |>
    dplyr::transmute(
      phenotype, arm,
      fs = if (identical(policy, "own")) {
        dplyr::if_else(has_time, fs, unname(seed_fs[phenotype]))
      } else {
        unname(seed_fs[phenotype])
      })
}

rate_map <- function(policy) {
  r <- rates_under(policy)
  split(setNames(r$fs, r$arm), r$phenotype)
}

disputed <- rates_under("own") |>
  dplyr::rename(fs_own = fs) |>
  dplyr::left_join(dplyr::rename(rates_under("seed"), fs_seed = fs),
    by = c("phenotype", "arm")) |>
  dplyr::filter(abs(fs_own - fs_seed) / fs_seed > 0.01)

nice_kable(rates_under("seed") |>
```

```
tidyr::pivot_wider(names_from = arm, values_from = fs),
digits = 1, caption = "Rate under the equivalent policy (Hz).")
```

Table 4: Rate under the equivalent policy (Hz).

phenotype	Real	Diffusion	GAN
Phenotype_A	10000	10000	10000
Phenotype_B	10000	10000	10000
Phenotype_C	10000	10000	10000
Phenotype_D	10000	10000	10000
Phenotype_E	20000	20000	20000

```
if (nrow(disputed)) {
  nice_kable(disputed, digits = 1,
             caption = "Arms where the two policies disagree. These are the only ones whose rate-depen
} else {
  cat("No arm's own time column disagrees with its seed; the two policies coincide.\n")
}
```

Table 5: Arms where the two policies disagree. These are the only ones whose rate-dependent metrics change between policies.

phenotype	arm	fs_own	fs_seed
Phenotype_E	Diffusion	10000	20000

```
FS_MAIN <- rate_map(CFG$rate_policy)
FS_ALT  <- rate_map(if (identical(CFG$rate_policy, "seed")) "own" else "seed")

# Which way openings go, per phenotype, read once off the real record's own
# labels and imposed on every arm. A recording convention rather than a tuned
# parameter, and the same bargain as taking one tcrit from the real record.
POL_MAIN <- inventory |>
  dplyr::filter(arm == "Real") |>
  dplyr::transmute(phenotype, polarity = sign(unit_amp))
POL_MAIN <- setNames(POL_MAIN$polarity, POL_MAIN$phenotype)

if (!all(POL_MAIN %in% c(-1, 1))) {
  stop("A real record has no measurable unitary amplitude, so the direction ",
       "of its openings is undefined: ",
       paste(names(POL_MAIN)[!(POL_MAIN %in% c(-1, 1))], collapse = ", "))
}
```

The lag-1 autocorrelation of the single-level noise, reported after the analysis, is the physical evidence: decimation decorrelates adjacent samples, so a record genuinely at half the rate sits well below its own seed while unaffected phenotypes match.

Noise level and segment length for the spectral metrics

The noise metrics need contiguous samples at one conductance level. Which level that can be depends on the record: a quiet channel offers long closed periods, a record at Popen 0.99 offers long open ones and closures only a sample or two wide. The level is therefore chosen per phenotype as whichever *every* arm of that phenotype can support, and reported so it can be stated in the legend.

Reporting an open-level spectrum is not a fudge: the requirement is that no gating transition falls inside a segment, which the open level satisfies exactly as well. It does measure something slightly different — open-channel noise sits on top of the instrumentation noise — but it is the same quantity in both arms of the comparison, which is all the paired metric needs.

The segment length is chosen per phenotype for the same reason the level is. Only the three arms within a phenotype are ever compared, so nothing requires A and E to share a segment length, and making them share one lets the single worst record dictate the resolution for every other. What *is* required is that the three arms of one phenotype share it, since the RMS distance is taken between their spectra.

The "auto" rule also runs the other way from the obvious one. Taking the largest segment a record tolerates maximises resolution and minimises the number of segments, which is the wrong trade for a statistic whose variance falls as $1/n_{\text{seg}}$. So "auto" takes the largest power of two that still gives every arm `psd_n_seg_min` segments, and only falls back towards the resolution floor when no length manages it.

```
cap_at <- inventory |>
  dplyr::group_by(phenotype) |>
  dplyr::summarise(closed = min(longest_closed), open = min(longest_open),
    .groups = "drop")

levels_used <- cap_at |>
  dplyr::mutate(
    level = if (identical(CFG$noise_level, "auto")) {
      dplyr::if_else(open > closed, "open", "closed")
    } else {
      CFG$noise_level
    },
    cap = dplyr::if_else(level == "open", open, closed))

if (any(levels_used$cap == 0)) {
  z <- levels_used |> dplyr::filter(cap == 0)
  stop("No usable ", paste(unique(z$level), collapse = "/"), " segments at all in: ",
    paste(z$phenotype, collapse = ", "),
    ". Set CFG$noise_level to the other level, or exclude these phenotypes.")
}

nice_kable(levels_used, digits = 0,
  caption = "Longest run every arm can supply at each level, and the level used. 'cap' is the")
```

Table 6: Longest run every arm can supply at each level, and the level used. 'cap' is that phenotype's ceiling on segment length.

phenotype	closed	open	level	cap
Phenotype_A	1533	426	closed	1533
Phenotype_B	4668	520	closed	4668
Phenotype_C	3597	1855	closed	3597
Phenotype_D	6	950	open	950
Phenotype_E	149	579	open	579

```
LEVEL_OF <- setNames(levels_used$level, levels_used$phenotype)
CAP_OF <- setNames(levels_used$cap, levels_used$phenotype)

# Run lengths at the chosen level, per arm, per phenotype. Everything below is
# decided from these, before a single spectrum is computed.
```

```

lens_of <- purrr::imap(recs, function(arms, ph)
  lapply(arms, run_lengths_at, level = LEVEL_OF[[ph]]))

seg_counts_at <- function(lens, nperseg)
  vapply(lens, seg_count, integer(1), nperseg = nperseg)

# The floor on segment length: coarser than psd_seg_min is not worth reporting,
# and the ACF needs segments longer than its own maximum lag.
SEG_FLOOR <- as.integer(2^ceiling(log2(max(CFG$psd_seg_min, CFG$acf_lag_max + 1))))

pick_nperseg <- function(lens, cap) {
  hi <- 2^floor(log2(cap))
  if (hi < SEG_FLOOR) return(NA_integer_)
  for (nps in 2^(floor(log2(hi)):floor(log2(SEG_FLOOR)))) {
    if (min(seg_counts_at(lens, nps)) >= CFG$psd_n_seg_min) return(as.integer(nps))
  }
  SEG_FLOOR
}

if (identical(CFG$psd_seg, "auto")) {
  NPERSEG_OF <- vapply(names(recs),
    function(ph) pick_nperseg(lens_of[[ph]], CAP_OF[[ph]]),
    integer(1))
  starved <- names(NPERSEG_OF)[is.na(NPERSEG_OF)]
  if (length(starved)) {
    stop(paste(starved, collapse = ", "), ": no arm supports ", SEG_FLOOR,
      " contiguous samples at the chosen level (caps ",
      paste(CAP_OF[starved], collapse = ", "), "). A spectrum that coarse ",
      "is not worth reporting. Exclude the phenotype with CFG$only, or ",
      "lower psd_seg_min and acf_lag_max deliberately and say so in the ",
      "methods.")
  }
} else {
  nps <- as.integer(CFG$psd_seg)
  if (nps < CFG$psd_seg_min) {
    stop("CFG$psd_seg = ", nps, " is below CFG$psd_seg_min = ", CFG$psd_seg_min, ".")
  }
  if (nps <= CFG$acf_lag_max) {
    stop("CFG$psd_seg = ", nps, " is not longer than CFG$acf_lag_max = ",
      CFG$acf_lag_max, ", so the ACF would be estimated over runs too short ",
      "to support it. Lower acf_lag_max.")
  }
  short <- names(CAP_OF)[CAP_OF < nps]
  if (length(short)) {
    stop("CFG$psd_seg = ", nps, " but ", paste(short, collapse = ", "),
      " supports only ", paste(CAP_OF[short], collapse = ", "),
      " samples at the chosen level. Set CFG$psd_seg = \"auto\".")
  }
  NPERSEG_OF <- setNames(rep(nps, length(recs)), names(recs))
}

seg_tbl <- purrr::imap_dfr(lens_of, function(lens, ph) {
  n <- seg_counts_at(lens, NPERSEG_OF[[ph]])

```

```

tibble::tibble(
  phenotype = ph, level = LEVEL_OF[[ph]], cap = CAP_OF[[ph]],
  nperseg = NPERSEG_OF[[ph]],
  seg_ms = 1000 * NPERSEG_OF[[ph]] / FS_MAIN[[ph]][["Real"]],
  res_hz = FS_MAIN[[ph]][["Real"]] / NPERSEG_OF[[ph]],
  Real = unname(n["Real"]), Diffusion = unname(n["Diffusion"]),
  GAN = if ("GAN" %in% names(n)) unname(n["GAN"]) else NA_integer_,
  n_used = min(n))
})

nice_kable(seg_tbl, digits = 1,
  caption = paste("Segment length chosen per phenotype, and the",
    "segments each arm can supply at it. n_used is",
    "what every arm is held to when",
    "equalise_noise is TRUE.))")

```

Table 7: Segment length chosen per phenotype, and the segments each arm can supply at it. n_used is what every arm is held to when equalise_noise is TRUE.

phenotype	level	cap	nperseg	seg_ms	res_hz	Real	Diffusion	GAN	n_used
Phenotype_A	closed	1533	512	51.2	19.5	36	29	45	29
Phenotype_B	closed	4668	1024	102.4	9.8	120	113	85	85
Phenotype_C	closed	3597	1024	102.4	9.8	55	26	40	26
Phenotype_D	open	950	256	25.6	39.1	422	415	179	179
Phenotype_E	open	579	256	12.8	78.1	86	94	94	86

```

readr::write_csv(seg_tbl, file.path(CFG$out_dir, "psd_segments.csv"))

thin <- seg_tbl |> dplyr::filter(n_used < CFG$psd_n_seg_min)
if (nrow(thin)) {
  warning("Fewer than CFG$psd_n_seg_min = ", CFG$psd_n_seg_min,
    " segments per arm in: ",
    paste(sprintf("%s (%d)", thin$phenotype, thin$n_used), collapse = ", "),
    ". Welch's variance goes as 1/n_seg, so those spectra are noisy and ",
    "their PSD RMS is not a reliable measure of spectral mismatch. The ",
    "equalisation makes the arms comparable with each other but cannot ",
    "manufacture precision. Flagged as psd_n_seg_ok in the provenance ",
    "table; say so in the methods or drop the phenotype from the ",
    "spectral metrics.")
}

if (dplyr::n_distinct(levels_used$level) > 1 ||
  dplyr::n_distinct(NPERSEG_OF) > 1) {
  cat("Level and/or segment length differ between phenotypes, so the noise\n",
    "metrics are paired within a phenotype but not comparable in absolute\n",
    "terms across them. State both per panel in the figure legend.\n", sep = "\n")
}

```

```

## Level and/or segment length differ between phenotypes, so the noise
## metrics are paired within a phenotype but not comparable in absolute
## terms across them. State both per panel in the figure legend.

```

3. Per-phenotype analysis

Each phenotype is reduced to a long table of metrics, plus the curves needed for the pooled figures. `direction` records which way is better, because pooling a distance with a summary statistic that should *match* the real record would be meaningless otherwise.

```
analyse_one <- function(row, arms_all, fs, level, nperseg) {
  ph <- row$phenotype

  real    <- arms_all$Real
  arms    <- arms_all[setdiff(names(arms_all), "Real")]
  has_gan <- "GAN" %in% names(arms)

  # ---- windows and MMD -----
  W <- list(Real = make_windows(real$current, CFG$seq_len, CFG$n_windows_mmd))
  for (a in names(arms)) W[[a]] <- make_windows(arms[[a]]$current, CFG$seq_len,
                                                CFG$n_windows_mmd)

  half <- floor(nrow(real) / 2)
  W_r1 <- make_windows(real$current[1:half], CFG$seq_len, CFG$n_windows_mmd)
  W_r2 <- make_windows(real$current[(half + 1):nrow(real)], CFG$seq_len, CFG$n_windows_mmd)
  W_wn <- make_windows(rnorm(nrow(real), mean(real$current), sd(real$current)),
                        CFG$seq_len, CFG$n_windows_mmd)

  # One bandwidth for every comparison in this phenotype, or the numbers are
  # not on a single scale.
  gam <- median_gamma(W$Real, W$Diffusion)

  # ---- single-level noise -----
  # Runs are kept separate and every record has already been shown to hold at
  # least one segment, so there is no missing-data branch here.
  runs <- lapply(arms_all, level_runs, level = level, min_len = nperseg)

  # Equal row counts do not give equal usable material: an arm that gates more
  # often has shorter runs and so fewer segments. Both noise metrics are RMS
  # differences between estimates, so the noisier arm is penalised for its
  # gating twice, once in the dwell metrics and again here. Holding every arm
  # to the counts the thinnest can supply makes that variance common.
  lens      <- lapply(runs, function(x) vapply(x, length, integer(1)))
  seg_avail <- vapply(lens, seg_count, integer(1), nperseg = nperseg)
  samp_avail <- vapply(lens, function(L) sum(L[L > CFG$acf_lag_max]), integer(1))
  seg_cap   <- if (isTRUE(CFG$equalise_noise)) min(seg_avail) else Inf
  samp_cap  <- if (isTRUE(CFG$equalise_noise)) min(samp_avail) else Inf

  psd <- purrr::imap_dfr(runs, function(x, a)
    dplyr::mutate(welch_psd(x, fs[[a]], nperseg, seg_cap), source = a))
  acf_tbl <- purrr::imap_dfr(runs, function(x, a)
    dplyr::mutate(acf_of(x, CFG$acf_lag_max, fs[[a]], samp_cap), source = a))

  # Lag-1 autocorrelation of the single-level noise, per arm. A record genuinely
  # at a lower rate than the one assumed for it sits well below its own seed.
  lag1 <- acf_tbl |> dplyr::filter(lag_n == 2)
  lag1 <- setNames(lag1$acf, lag1$source)

  # Two arms at different rates have different frequency and lag grids, so both
```

```

# comparisons interpolate onto a shared axis rather than pivoting on the grid
# value. Under the equivalent policy the grids already coincide and the
# interpolation is a no-op.
curve <- function(tbl, a, xvar, yvar) {
  d <- tbl[tbl$source == a, ]
  list(x = d[[xvar]], y = d[[yvar]])
}

psd_rms <- function(a) {
  r <- curve(psd, "Real", "freq", "psd")
  g <- curve(psd, a, "freq", "psd")
  hi <- min(CFG$filter_hz, max(r$x), max(g$x))
  lo <- max(min(r$x), min(g$x))
  grid <- exp(seq(log(lo), log(hi), length.out = 512))
  rms_diff(approx(r$x, log10(r$y), grid)$y, approx(g$x, log10(g$y), grid)$y)
}

acf_rms <- function(a) {
  r <- curve(acf_tbl, "Real", "lag_ms", "acf")
  g <- curve(acf_tbl, a, "lag_ms", "acf")
  grid <- seq(0, min(max(r$x), max(g$x)), length.out = CFG$acf_lag_max + 1)
  rms_diff(approx(r$x, r$y, grid)$y, approx(g$x, g$y, grid)$y)
}

# ---- slow envelope -----
env <- list(Real = block_mean(real$current, CFG$slow_factor))
for (a in names(arms)) env[[a]] <- block_mean(arms[[a]]$current, CFG$slow_factor)
We <- lapply(env, function(e) make_windows(e, min(64, length(e)), CFG$n_windows_mmd))

# ---- dwell times -----
dw <- purrr::imap_dfr(arms_all, function(d, a)
  dplyr::mutate(dwell_times(d$channels, fs[[a]]), source = a))
mean_dwell <- function(a, s) {
  v <- dw |> dplyr::filter(source == a, state == s) |> dplyr::pull(dwell_ms)
  if (!length(v)) stop(ph, "/", a, ": no ", s, " dwells to average")
  mean(v)
}

level_pts <- function(d) d$current[at_level(d, level)]

# ---- assemble -----
# One metric name whatever the level, so the pooled comparison keeps all
# phenotypes; the level itself is recorded in the provenance table.
sd_name <- "Noise SD (pA)"
M <- function(metric, method, direction, value)
  tibble::tibble(phenotype = ph, metric = metric, method = method,
    direction = direction, value = as.numeric(value))

rows <- dplyr::bind_rows(
  M("MMD vs real", "Real floor", "reference", mmd_rbf(W_r1, W_r2, gamma = gam)),
  M("MMD vs real", "White noise", "reference", mmd_rbf(W$Real, W_wn, gamma = gam)),
  M(sd_name, "Real", "reference", sd(level_pts(real))),
  M("Envelope SD (pA)", "Real", "reference", sd(env$Real)),

```



```

M("Open probability", "Real", "reference", mean(real$channels > 0)),
M("Mean open dwell (ms)", "Real", "reference", mean_dwell("Real", "Open")),
M("Mean closed dwell (ms)", "Real", "reference", mean_dwell("Real", "Closed")),
purrr::map_dfr(names(arms), function(a) {
  pm <- mmd_permutation_p(W$Real, W[[a]], gamma = gam)
  dplyr::bind_rows(
    M("MMD vs real", a, "lower", mmd_rbf(W$Real, W[[a]], gamma = gam)),
    M("MMD permutation p", a, "higher", pm$p),
    M("MMD z-score", a, "lower", pm$z),
    M("KS amplitude", a, "lower",
      unname(ks_safe(real$current, arms[[a]]$current)$statistic)),
    M("log10 PSD RMS", a, "lower", psd_rms(a)),
    M("ACF RMS", a, "lower", acf_rms(a)),
    M("MMD envelope", a, "lower", mmd_rbf(We$Real, We[[a]])),
    M(sd_name, a, "match_real", sd(level_pts(arms[[a]]))),
    M("Envelope SD (pA)", a, "match_real", sd(env[[a]])),
    M("Open probability", a, "match_real", mean(arms[[a]]$channels > 0)),
    M("Mean open dwell (ms)", a, "match_real", mean_dwell(a, "Open")),
    M("Mean closed dwell (ms)", a, "match_real", mean_dwell(a, "Closed"))
  )
})
)

# ---- curves and provenance -----
# time_ms is built from the sample index and the rate assigned to the arm, not
# from the file's own time column, which for a DDPM export is a fixed 1e-4
# step regardless of what it was trained on and makes panels differ in width.
traces <- purrr::imap_dfr(arms_all, function(d, a) {
  n <- min(nrow(d), round(CFG$trace_secs * fs[[a]]))
  s <- best_window(d$channels, n)
  d[s:(s + n - 1), ] |>
    dplyr::mutate(source = a, phenotype = ph,
      time_ms = 1000 * (dplyr::row_number() - 1) / fs[[a]],
      win_start = s, win_popen = mean(channels > 0))
})

amp <- purrr::imap_dfr(arms_all, function(d, a)
  tibble::tibble(current = d$current, source = a, phenotype = ph))

info <- tibble::tibble(
  phenotype = ph,
  seed_file = basename(row$raw),
  gan_file = if (has_gan) basename(row$gan) else "none",
  ddpm_file = basename(row$ddpm),
  n_real = nrow(real),
  n_ddpm = nrow(arms$Diffusion),
  n_gan = if (has_gan) nrow(arms$GAN) else NA_integer_,
  popen_real = mean(real$channels > 0),
  max_level_real = max(real$channels),
  fs_real = fs[["Real"]],
  fs_ddpm = fs[["Diffusion"]],
  fs_gan = if (has_gan) fs[["GAN"]] else NA_real_,
  noise_level = level,

```



```

psd_nperseg = nperseg,
psd_seg_ms = 1000 * nperseg / fs[["Real"]],
# Segments each arm could have supplied, then the count they were all held
# to. Indexed by name rather than position: Real is first by construction
# today, which is not a reason to rely on it.
psd_n_seg_real = unname(seg_avail["Real"]),
psd_n_seg_ddpm = unname(seg_avail["Diffusion"]),
psd_n_seg_gan = if (has_gan) unname(seg_avail["GAN"]) else NA_integer_,
psd_n_seg_used = psd$n_seg[psd$source == "Real"][1],
psd_n_seg_ok = min(seg_avail) >= CFG$psd_n_seg_min,
acf_n_used = acf_tbl$n_used[acf_tbl$source == "Real"][1],
lag1_real = unname(lag1["Real"]),
lag1_ddpm = unname(lag1["Diffusion"]),
lag1_gan = if (has_gan) unname(lag1["GAN"]) else NA_real_,
gamma = gam
)

list(metrics = rows,
      psd = dplyr::mutate(psd, phenotype = ph),
      acf = dplyr::mutate(acf_tbl, phenotype = ph),
      dwell = dplyr::mutate(dw, phenotype = ph),
      env = purrr::imap_dfr(env, function(e, a)
        tibble::tibble(idx = seq_along(e), value = e, source = a, phenotype = ph)),
      traces = traces,
      amp = amp,
      info = info)
}

# The cache is keyed on the segment length, whether the arms were equalised,
# the rates in force and the policy label: results computed under different
# assumptions are not comparable with these and must not be silently reused.
run_one <- function(row, fs, policy) {
  ph <- row$phenotype
  nps <- NPERSEG_OF[[ph]]
  key <- sprintf("%s_seg%d%s_%s_%s_%s", ph, nps, LEVEL_OF[[ph]], policy,
    if (isTRUE(CFG$equalise_noise)) "eq" else "raw",
    paste(sprintf("%s%g", substr(names(fs), 1, 1), fs), collapse = ""))
  cache_file <- file.path(CFG$out_dir, "cache", paste0(key, ".rds"))
  if (CFG$cache && file.exists(cache_file)) {
    message("Reusing cached ", key)
    return(readRDS(cache_file))
  }
  message("Analysing ", ph, " (", policy, " rates)")
  res <- analyse_one(row, recs[[ph]], fs, LEVEL_OF[[ph]], nps)
  if (CFG$cache) saveRDS(res, cache_file)
  res
}

results <- purrr::map(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  run_one(row, FS_MAIN[[row$phenotype]], CFG$rate_policy)
})
names(results) <- manifest$phenotype

```

```

pull_all <- function(what) purrr::map_dfr(results, what)

metrics <- pull_all("metrics")
info <- pull_all("info")

# Nothing missing gets past this point. An NA here would become a phenotype
# silently dropped from a paired comparison, which is worse than a failed knit.
bad <- dplyr::filter(metrics, !is.finite(value))
if (nrow(bad)) {
  stop("Non-finite metrics, which must be understood rather than dropped:\n",
       paste(sprintf(" %s / %s / %s", bad$phenotype, bad$method, bad$metric),
             collapse = "\n"))
}

nice_kable(
  info |> dplyr::select(phenotype, seed_file, ddpm_file, gan_file,
                      n_real, n_ddpm, n_gan, popen_real, max_level_real),
  digits = 4, caption = "Records analysed."
)

```

Table 8: Records analysed.

phenotype	seed_file	ddpm_file	gan_file	n
Phenotype_A	4096PYRraw.csv	Phenotype_A_DDPM.parquet	4096PYRGAN.csv	7
Phenotype_B	4096edr01raw.csv	Phenotype_B_DDPM.parquet	4096edr01GAN.csv	10
Phenotype_C	4096lina11raw.csv	Phenotype_C_DDPM.parquet	4096Lina2GAN_data.csv	10
Phenotype_D	pvnraw.csv	Phenotype_D_DDPM.parquet	pvn_GAN_512butt.parquet	10
Phenotype_E	c035[NF=50Hz]0-32_4kraw.csv	Phenotype_E_DDPM.parquet	c035[NF=50Hz]0-32_4kGAN.csv	10

```

nice_kable(info |> dplyr::select(phenotype, noise_level, fs_real, fs_ddpm,
                                fs_gan, psd_nperseg, psd_seg_ms,
                                psd_n_seg_real, psd_n_seg_ddpm,
                                psd_n_seg_gan, psd_n_seg_used,
                                psd_n_seg_ok, acf_n_used, gamma),
  digits = 4,
  caption = paste("Rate applied, spectral segment length, segments",
                  "each arm could supply, the count they were all",
                  "held to, and kernel bandwidth. psd_n_seg_ok is",
                  "FALSE where the spectra are too thinly averaged",
                  "to be worth much, however well equalised."))

```

Table 9: Rate applied, spectral segment length, segments, each arm could supply, the count they were all held to, and psd_n_seg_ok is FALSE where the spectra are too thinly averaged to be worth much, however well equalised.

phenotype	noise_level	fs_real	fs_ddpm	fs_gan	psd_nperseg	psd_seg_ms	psd_n_seg_real	psd_n_seg
Phenotype_A	closed	10000	10000	10000	512	51.2	36	
Phenotype_B	closed	10000	10000	10000	1024	102.4	120	
Phenotype_C	closed	10000	10000	10000	1024	102.4	55	
Phenotype_D	open	10000	10000	10000	256	25.6	422	

Phenotype_E	open	20000	20000	20000	256	12.8	86
-------------	------	-------	-------	-------	-----	------	----

```
nice_kable(info |> dplyr::select(phenotype, lag1_real, lag1_ddpm, lag1_gan),
  digits = 4,
  caption = "Lag-1 autocorrelation of the single-level noise. An arm well below its own real
```

Table 10: Lag-1 autocorrelation of the single-level noise. An arm well below its own real value is genuinely at a lower rate than the one assumed for it, not merely mislabelled.

phenotype	lag1_real	lag1_ddpm	lag1_gan
Phenotype_A	0.6772	0.7224	0.5453
Phenotype_B	0.0905	0.1414	0.2308
Phenotype_C	0.0930	0.0214	0.3993
Phenotype_D	0.8880	0.8703	0.8765
Phenotype_E	0.8371	0.8294	0.8086

```
readr::write_csv(info, file.path(CFG$out_dir, "provenance.csv"))
readr::write_csv(metrics, file.path(CFG$out_dir, "metrics_long.csv"))
```

Equivalent versus honest

Only rate-dependent metrics can move between the policies. The amplitude metrics, MMD and open probability are computed per sample and are identical either way, which is itself useful: it means the choice cannot be dodged by appealing to the metrics that happen to be convenient.

```
alt_phenos <- unique(disputed$phenotype)

if (!length(alt_phenos)) {
  cat("No arm's rate is disputed, so the two policies give identical results.\n")
} else {
  alt <- purrr::map(alt_phenos, function(ph) {
    row <- manifest[manifest$phenotype == ph, ]
    run_one(row, FS_ALT[[ph]], if (identical(CFG$rate_policy, "seed")) "own" else "seed")
  })
  names(alt) <- alt_phenos

  rate_sensitive <- c("log10 PSD RMS", "ACF RMS",
    "Mean open dwell (ms)", "Mean closed dwell (ms)")

  cmp <- dplyr::bind_rows(
    metrics |> dplyr::filter(phenotype %in% alt_phenos) |>
      dplyr::mutate(policy = CFG$rate_policy),
    purrr::map_dfr(alt, "metrics") |>
      dplyr::mutate(policy = if (identical(CFG$rate_policy, "seed")) "own" else "seed")
  ) |>
    dplyr::filter(metric %in% rate_sensitive) |>
    dplyr::mutate(policy = dplyr::recode(policy, seed = "equivalent", own = "honest")) |>
    dplyr::select(phenotype, metric, method, policy, value) |>
    tidyr::pivot_wider(names_from = policy, values_from = value) |>
    dplyr::mutate(ratio = honest / equivalent)
```

```

nice_kable(cmp, digits = 4,
  caption = "Rate-dependent metrics under both readings. Everything else is unchanged by c
readr::write_csv(cmp, file.path(CFG$out_dir, "rate_policy_comparison.csv"))

lag_cmp <- dplyr::bind_rows(
  info |> dplyr::filter(phenotype %in% alt_phenos) |>
    dplyr::mutate(policy = CFG$rate_policy),
  purrr::map_dfr(alt, "info") |>
    dplyr::mutate(policy = if (identical(CFG$rate_policy, "seed")) "own" else "seed")
) |>
  dplyr::select(phenotype, policy, lag1_real, lag1_ddpm, lag1_gan)
nice_kable(lag_cmp, digits = 4,
  caption = "Lag-1 autocorrelation, which does not depend on the rate label at all and is
}

```

Table 11: Lag-1 autocorrelation, which does not depend on the rate label at all and is therefore the evidence.

phenotype	policy	lag1_real	lag1_ddpm	lag1_gan
Phenotype_E	seed	0.8371	0.8294	0.8086
Phenotype_E	own	0.8371	0.8294	0.8086

How to read this

The lag-1 table is the part that decides it, because it is computed per sample and is untouched by whichever label is applied. If the disputed arm’s value sits close to its own seed record, the samples are correlated like the seed’s are and the record is a sample-for-sample stand-in with a cosmetic time column: **equivalent** is right, and the honest reading inflates its dwell times by the rate ratio for no physical reason. If it sits well below, adjacent samples really are less correlated, the record was produced at the lower rate, and **honest** is right — but then the record has less bandwidth than the seed, which is a genuine limitation of that arm and belongs in the results rather than being corrected away.

The dwell ratios are the sanity check on whichever answer you accept: under the honest reading a 5 kHz record’s mean dwells double exactly, so if the honest dwell times land implausibly far from the seed’s while the equivalent ones sit close, that is evidence for the equivalent reading and vice versa.

4. Scoring

Distances should be small, the permutation p should be large, and summary statistics such as the noise SD should *match* the real record rather than approach zero. The score column puts all three on a “smaller is better” footing except the p, which is flagged separately.

```

refs <- metrics |>
  dplyr::filter(direction == "reference", method == "Real") |>
  dplyr::select(phenotype, metric, ref = value)

scored <- metrics |>
  dplyr::filter(direction %in% c("lower", "higher", "match_real")) |>
  dplyr::left_join(refs, by = c("phenotype", "metric")) |>
  dplyr::mutate(
    score = dplyr::case_when(
      direction == "match_real" ~ abs(value - ref),
      direction == "higher" ~ 1 - value,

```

```

    TRUE ~ value
  ),
  better = "lower"
) |>
dplyr::filter(is.finite(score))

nice_kable(
  scored |> dplyr::arrange(metric, method, phenotype) |>
  dplyr::select(metric, method, phenotype, value, ref, score),
  digits = 5, caption = "Raw values and the comparable score derived from them."
)

```

Table 12: Raw values and the comparable score derived from them.

metric	method	phenotype	value	ref	score
ACF RMS	Diffusion	Phenotype_A	0.06887	NA	0.06887
ACF RMS	Diffusion	Phenotype_B	0.01165	NA	0.01165
ACF RMS	Diffusion	Phenotype_C	0.01909	NA	0.01909
ACF RMS	Diffusion	Phenotype_D	0.03880	NA	0.03880
ACF RMS	Diffusion	Phenotype_E	0.01557	NA	0.01557
ACF RMS	GAN	Phenotype_A	0.09234	NA	0.09234
ACF RMS	GAN	Phenotype_B	0.03017	NA	0.03017
ACF RMS	GAN	Phenotype_C	0.14639	NA	0.14639
ACF RMS	GAN	Phenotype_D	0.03908	NA	0.03908
ACF RMS	GAN	Phenotype_E	0.02630	NA	0.02630
Envelope SD (pA)	Diffusion	Phenotype_A	3.47587	3.63859	0.16273
Envelope SD (pA)	Diffusion	Phenotype_B	2.36635	2.39297	0.02662
Envelope SD (pA)	Diffusion	Phenotype_C	0.20030	0.22258	0.02229
Envelope SD (pA)	Diffusion	Phenotype_D	0.04133	0.05290	0.01156
Envelope SD (pA)	Diffusion	Phenotype_E	1.72760	2.40546	0.67785
Envelope SD (pA)	GAN	Phenotype_A	3.92811	3.63859	0.28952
Envelope SD (pA)	GAN	Phenotype_B	3.53319	2.39297	1.14022
Envelope SD (pA)	GAN	Phenotype_C	0.20660	0.22258	0.01598
Envelope SD (pA)	GAN	Phenotype_D	0.04825	0.05290	0.00464
Envelope SD (pA)	GAN	Phenotype_E	1.53494	2.40546	0.87052
KS amplitude	Diffusion	Phenotype_A	0.05748	NA	0.05748
KS amplitude	Diffusion	Phenotype_B	0.01977	NA	0.01977
KS amplitude	Diffusion	Phenotype_C	0.28494	NA	0.28494
KS amplitude	Diffusion	Phenotype_D	0.07664	NA	0.07664
KS amplitude	Diffusion	Phenotype_E	0.07343	NA	0.07343
KS amplitude	GAN	Phenotype_A	0.08907	NA	0.08907
KS amplitude	GAN	Phenotype_B	0.12302	NA	0.12302
KS amplitude	GAN	Phenotype_C	0.14777	NA	0.14777
KS amplitude	GAN	Phenotype_D	0.02207	NA	0.02207
KS amplitude	GAN	Phenotype_E	0.13810	NA	0.13810
MMD envelope	Diffusion	Phenotype_A	0.00924	NA	0.00924
MMD envelope	Diffusion	Phenotype_B	0.00003	NA	0.00003
MMD envelope	Diffusion	Phenotype_C	0.04906	NA	0.04906
MMD envelope	Diffusion	Phenotype_D	0.04053	NA	0.04053
MMD envelope	Diffusion	Phenotype_E	0.00969	NA	0.00969

MMD envelope	GAN	Phenotype_A	0.02272	NA	0.02272
MMD envelope	GAN	Phenotype_B	0.08477	NA	0.08477
MMD envelope	GAN	Phenotype_C	0.07580	NA	0.07580
MMD envelope	GAN	Phenotype_D	0.00796	NA	0.00796
MMD envelope	GAN	Phenotype_E	0.01903	NA	0.01903
MMD permutation p	Diffusion	Phenotype_A	0.42289	NA	0.57711
MMD permutation p	Diffusion	Phenotype_B	0.21393	NA	0.78607
MMD permutation p	Diffusion	Phenotype_C	0.00498	NA	0.99502
MMD permutation p	Diffusion	Phenotype_D	0.00498	NA	0.99502
MMD permutation p	Diffusion	Phenotype_E	0.00498	NA	0.99502
MMD permutation p	GAN	Phenotype_A	0.05473	NA	0.94527
MMD permutation p	GAN	Phenotype_B	0.00498	NA	0.99502
MMD permutation p	GAN	Phenotype_C	0.00498	NA	0.99502
MMD permutation p	GAN	Phenotype_D	0.00995	NA	0.99005
MMD permutation p	GAN	Phenotype_E	0.00995	NA	0.99005
MMD vs real	Diffusion	Phenotype_A	-0.00039	NA	-0.00039
MMD vs real	Diffusion	Phenotype_B	0.00030	NA	0.00030
MMD vs real	Diffusion	Phenotype_C	0.06394	NA	0.06394
MMD vs real	Diffusion	Phenotype_D	0.01014	NA	0.01014
MMD vs real	Diffusion	Phenotype_E	0.00478	NA	0.00478
MMD vs real	GAN	Phenotype_A	0.00381	NA	0.00381
MMD vs real	GAN	Phenotype_B	0.00991	NA	0.00991
MMD vs real	GAN	Phenotype_C	0.04871	NA	0.04871
MMD vs real	GAN	Phenotype_D	0.00178	NA	0.00178
MMD vs real	GAN	Phenotype_E	0.00485	NA	0.00485
MMD z-score	Diffusion	Phenotype_A	-0.22404	NA	-0.22404
MMD z-score	Diffusion	Phenotype_B	0.52824	NA	0.52824
MMD z-score	Diffusion	Phenotype_C	28.81714	NA	28.81714
MMD z-score	Diffusion	Phenotype_D	21.71080	NA	21.71080
MMD z-score	Diffusion	Phenotype_E	5.54256	NA	5.54256
MMD z-score	GAN	Phenotype_A	1.79339	NA	1.79339
MMD z-score	GAN	Phenotype_B	20.35707	NA	20.35707
MMD z-score	GAN	Phenotype_C	17.07841	NA	17.07841
MMD z-score	GAN	Phenotype_D	3.31306	NA	3.31306
MMD z-score	GAN	Phenotype_E	5.33642	NA	5.33642
Mean closed dwell (ms)	Diffusion	Phenotype_A	3.95736	3.40100	0.55636
Mean closed dwell (ms)	Diffusion	Phenotype_B	161.57895	197.54468	35.96573
Mean closed dwell (ms)	Diffusion	Phenotype_C	25.54062	52.14079	26.60016
Mean closed dwell (ms)	Diffusion	Phenotype_D	0.39318	0.37929	0.01389
Mean closed dwell (ms)	Diffusion	Phenotype_E	0.80298	1.09652	0.29355
Mean closed dwell (ms)	GAN	Phenotype_A	4.08282	3.40100	0.68182
Mean closed dwell (ms)	GAN	Phenotype_B	112.10533	197.54468	85.43935
Mean closed dwell (ms)	GAN	Phenotype_C	16.53537	52.14079	35.60542
Mean closed dwell (ms)	GAN	Phenotype_D	0.22012	0.37929	0.15917
Mean closed dwell (ms)	GAN	Phenotype_E	0.52013	1.09652	0.57639
Mean open dwell (ms)	Diffusion	Phenotype_A	5.43705	4.75050	0.68655
Mean open dwell (ms)	Diffusion	Phenotype_B	14.10714	15.55217	1.44503
Mean open dwell (ms)	Diffusion	Phenotype_C	78.62604	79.43816	0.81212
Mean open dwell (ms)	Diffusion	Phenotype_D	32.07435	31.88000	0.19435
Mean open dwell (ms)	Diffusion	Phenotype_E	4.81500	4.89868	0.08368

Mean open dwell (ms)	GAN	Phenotype_A	4.90699	4.75050	0.15648
Mean open dwell (ms)	GAN	Phenotype_B	21.51486	15.55217	5.96269
Mean open dwell (ms)	GAN	Phenotype_C	15.61897	79.43816	63.81919
Mean open dwell (ms)	GAN	Phenotype_D	14.33624	31.88000	17.54376
Mean open dwell (ms)	GAN	Phenotype_E	3.82437	4.89868	1.07431
Noise SD (pA)	Diffusion	Phenotype_A	1.33296	1.43839	0.10543
Noise SD (pA)	Diffusion	Phenotype_B	1.20101	1.09713	0.10388
Noise SD (pA)	Diffusion	Phenotype_C	0.04001	0.04062	0.00061
Noise SD (pA)	Diffusion	Phenotype_D	0.09583	0.10893	0.01310
Noise SD (pA)	Diffusion	Phenotype_E	0.99899	1.01698	0.01799
Noise SD (pA)	GAN	Phenotype_A	1.51902	1.43839	0.08063
Noise SD (pA)	GAN	Phenotype_B	1.30958	1.09713	0.21245
Noise SD (pA)	GAN	Phenotype_C	0.05438	0.04062	0.01376
Noise SD (pA)	GAN	Phenotype_D	0.11360	0.10893	0.00466
Noise SD (pA)	GAN	Phenotype_E	1.16395	1.01698	0.14696
Open probability	Diffusion	Phenotype_A	0.57844	0.58251	0.00406
Open probability	Diffusion	Phenotype_B	0.07900	0.07154	0.00746
Open probability	Diffusion	Phenotype_C	0.75481	0.60373	0.15108
Open probability	Diffusion	Phenotype_D	0.98789	0.98828	0.00039
Open probability	Diffusion	Phenotype_E	0.85707	0.81710	0.03997
Open probability	GAN	Phenotype_A	0.54614	0.58251	0.03636
Open probability	GAN	Phenotype_B	0.15921	0.07154	0.08767
Open probability	GAN	Phenotype_C	0.48575	0.60373	0.11798
Open probability	GAN	Phenotype_D	0.98490	0.98828	0.00338
Open probability	GAN	Phenotype_E	0.88037	0.81710	0.06327
log10 PSD RMS	Diffusion	Phenotype_A	0.63556	NA	0.63556
log10 PSD RMS	Diffusion	Phenotype_B	0.17809	NA	0.17809
log10 PSD RMS	Diffusion	Phenotype_C	0.27779	NA	0.27779
log10 PSD RMS	Diffusion	Phenotype_D	0.16550	NA	0.16550
log10 PSD RMS	Diffusion	Phenotype_E	0.09052	NA	0.09052
log10 PSD RMS	GAN	Phenotype_A	0.60627	NA	0.60627
log10 PSD RMS	GAN	Phenotype_B	0.33314	NA	0.33314
log10 PSD RMS	GAN	Phenotype_C	0.81461	NA	0.81461
log10 PSD RMS	GAN	Phenotype_D	0.10500	NA	0.10500
log10 PSD RMS	GAN	Phenotype_E	0.12237	NA	0.12237

```
readr::write_csv(scored, file.path(CFG$out_dir, "scores_long.csv"))
```

Head to head, per phenotype

```
head_to_head <- scored |>
  dplyr::select(metric, phenotype, method, value) |>
  tidyr::pivot_wider(names_from = method, values_from = value) |>
  dplyr::left_join(refs, by = c("phenotype", "metric")) |>
  dplyr::rename(`Reference (real)` = ref) |>
  dplyr::arrange(metric, phenotype)

nice_kable(head_to_head, digits = 5,
            caption = paste("Similarity metrics, computed identically",
                            "for both methods. Lower is better throughout except",
```

"the permutation p, where a large value means the",
 "difference from real is no bigger than relabelling",
 "produces."))

Table 13: Similarity metrics, computed identically for both methods.
 Lower is better throughout except the permutation p, where a large
 value means the difference from real is no bigger than relabelling
 produces.

metric	phenotype	Diffusion	GAN	Reference (real)
ACF RMS	Phenotype_A	0.06887	0.09234	NA
ACF RMS	Phenotype_B	0.01165	0.03017	NA
ACF RMS	Phenotype_C	0.01909	0.14639	NA
ACF RMS	Phenotype_D	0.03880	0.03908	NA
ACF RMS	Phenotype_E	0.01557	0.02630	NA
Envelope SD (pA)	Phenotype_A	3.47587	3.92811	3.63859
Envelope SD (pA)	Phenotype_B	2.36635	3.53319	2.39297
Envelope SD (pA)	Phenotype_C	0.20030	0.20660	0.22258
Envelope SD (pA)	Phenotype_D	0.04133	0.04825	0.05290
Envelope SD (pA)	Phenotype_E	1.72760	1.53494	2.40546
KS amplitude	Phenotype_A	0.05748	0.08907	NA
KS amplitude	Phenotype_B	0.01977	0.12302	NA
KS amplitude	Phenotype_C	0.28494	0.14777	NA
KS amplitude	Phenotype_D	0.07664	0.02207	NA
KS amplitude	Phenotype_E	0.07343	0.13810	NA
MMD envelope	Phenotype_A	0.00924	0.02272	NA
MMD envelope	Phenotype_B	0.00003	0.08477	NA
MMD envelope	Phenotype_C	0.04906	0.07580	NA
MMD envelope	Phenotype_D	0.04053	0.00796	NA
MMD envelope	Phenotype_E	0.00969	0.01903	NA
MMD permutation p	Phenotype_A	0.42289	0.05473	NA
MMD permutation p	Phenotype_B	0.21393	0.00498	NA
MMD permutation p	Phenotype_C	0.00498	0.00498	NA
MMD permutation p	Phenotype_D	0.00498	0.00995	NA
MMD permutation p	Phenotype_E	0.00498	0.00995	NA
MMD vs real	Phenotype_A	-0.00039	0.00381	NA
MMD vs real	Phenotype_B	0.00030	0.00991	NA
MMD vs real	Phenotype_C	0.06394	0.04871	NA
MMD vs real	Phenotype_D	0.01014	0.00178	NA
MMD vs real	Phenotype_E	0.00478	0.00485	NA
MMD z-score	Phenotype_A	-0.22404	1.79339	NA
MMD z-score	Phenotype_B	0.52824	20.35707	NA
MMD z-score	Phenotype_C	28.81714	17.07841	NA
MMD z-score	Phenotype_D	21.71080	3.31306	NA
MMD z-score	Phenotype_E	5.54256	5.33642	NA
Mean closed dwell (ms)	Phenotype_A	3.95736	4.08282	3.40100
Mean closed dwell (ms)	Phenotype_B	161.57895	112.10533	197.54468
Mean closed dwell (ms)	Phenotype_C	25.54062	16.53537	52.14079
Mean closed dwell (ms)	Phenotype_D	0.39318	0.22012	0.37929
Mean closed dwell (ms)	Phenotype_E	0.80298	0.52013	1.09652

Mean open dwell (ms)	Phenotype_A	5.43705	4.90699	4.75050
Mean open dwell (ms)	Phenotype_B	14.10714	21.51486	15.55217
Mean open dwell (ms)	Phenotype_C	78.62604	15.61897	79.43816
Mean open dwell (ms)	Phenotype_D	32.07435	14.33624	31.88000
Mean open dwell (ms)	Phenotype_E	4.81500	3.82437	4.89868
Noise SD (pA)	Phenotype_A	1.33296	1.51902	1.43839
Noise SD (pA)	Phenotype_B	1.20101	1.30958	1.09713
Noise SD (pA)	Phenotype_C	0.04001	0.05438	0.04062
Noise SD (pA)	Phenotype_D	0.09583	0.11360	0.10893
Noise SD (pA)	Phenotype_E	0.99899	1.16395	1.01698
Open probability	Phenotype_A	0.57844	0.54614	0.58251
Open probability	Phenotype_B	0.07900	0.15921	0.07154
Open probability	Phenotype_C	0.75481	0.48575	0.60373
Open probability	Phenotype_D	0.98789	0.98490	0.98828
Open probability	Phenotype_E	0.85707	0.88037	0.81710
log10 PSD RMS	Phenotype_A	0.63556	0.60627	NA
log10 PSD RMS	Phenotype_B	0.17809	0.33314	NA
log10 PSD RMS	Phenotype_C	0.27779	0.81461	NA
log10 PSD RMS	Phenotype_D	0.16550	0.10500	NA
log10 PSD RMS	Phenotype_E	0.09052	0.12237	NA

```
readr::write_csv(head_to_head, file.path(CFG$out_dir, "head_to_head_metrics.csv"))
```

5. Pooled comparison

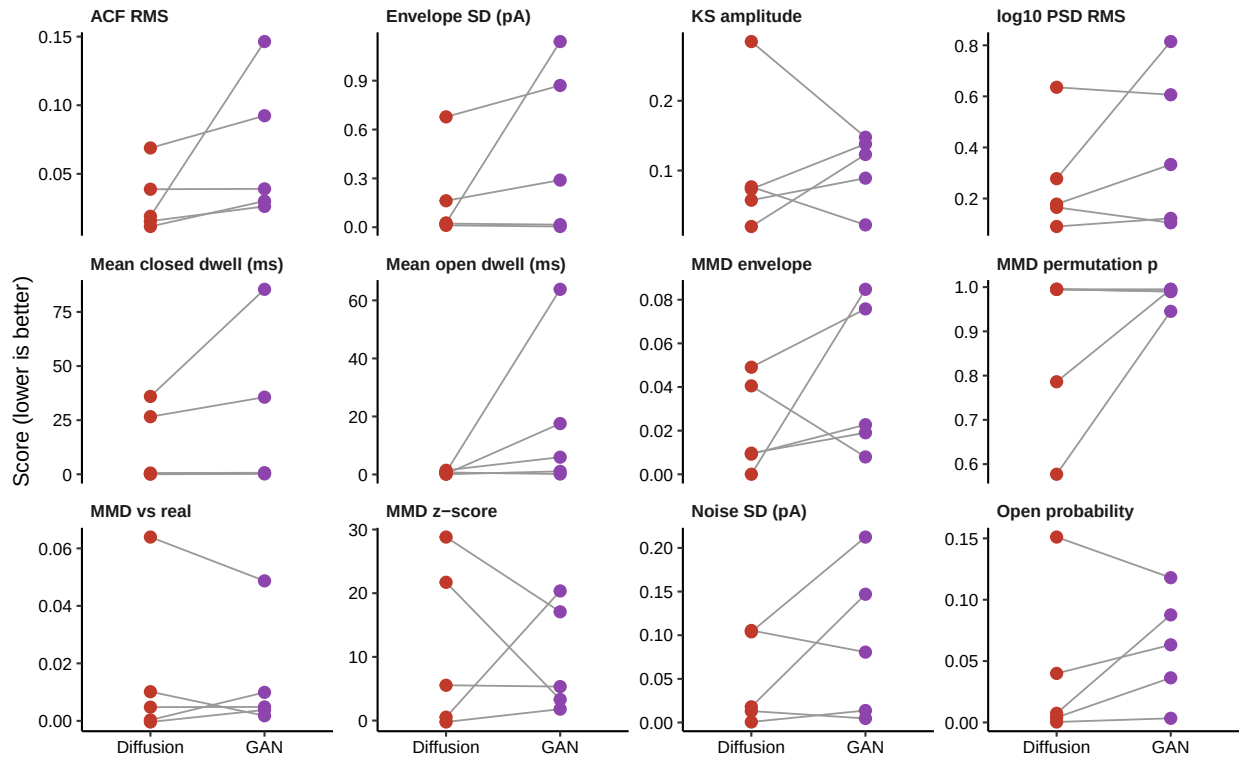
The design is paired: both methods see the same seed record, so the quantity of interest is the within-phenotype difference. Between-phenotype variation in noise, open probability and duration is far larger than any method effect and would swamp it if the records were treated as independent samples.

```
pairable <- scored |>
  dplyr::filter(method %in% c("Diffusion", "GAN")) |>
  dplyr::group_by(metric, phenotype) |>
  dplyr::filter(dplyr::n_distinct(method) == 2) |>
  dplyr::ungroup()

n_ph <- dplyr::n_distinct(pairable$phenotype)

p_slope <- ggplot(pairable, aes(factor(method, levels = c("Diffusion", "GAN")),
                                score, group = phenotype)) +
  geom_line(colour = "grey60", linewidth = 0.4) +
  geom_point(aes(colour = method), size = 2.2) +
  facet_wrap(~ metric, scales = "free_y") +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = NULL, y = "Score (lower is better)") +
  theme_channel()

p_slope
```



```
save_fig(p_slope, "fig_paired_metrics", width = 9, height = 6.5)
```

Each line is one phenotype. Lines that all slope the same way are the reportable result; lines that cross mean the ranking depends on which recording you happen to analyse.

```
wide <- pairable |>
  dplyr::select(metric, phenotype, better, method, score) |>
  tidyr::pivot_wider(names_from = method, values_from = score) |>
  dplyr::mutate(diff = Diffusion - GAN)

paired_stats <- wide |>
  dplyr::group_by(metric, better) |>
  dplyr::summarise(
    n = dplyr::n(),
    n_ties = sum(diff == 0),
    median_diff = median(diff),
    mean_diff = mean(diff),
    sd_diff = sd(diff),
    n_favour_diff = sum(if (dplyr::first(better) == "lower") diff < 0 else diff > 0),
    p_paired_t = if (dplyr::n() > 2 && sd(diff) > 0)
      stats::t.test(diff)$p.value else NA_real_,
    ci_lo = if (dplyr::n() > 2 && sd(diff) > 0)
      stats::t.test(diff)$conf.int[1] else NA_real_,
    ci_hi = if (dplyr::n() > 2 && sd(diff) > 0)
      stats::t.test(diff)$conf.int[2] else NA_real_,
    # The sign test must be taken in the same tail as `better`, and exact ties
    # are absence of evidence rather than evidence against: counting them as
    # unfavourable reported 0.0625 for a metric that was three-fifths tied.
    p_sign = {
```

```

    fav <- if (dplyr::first(better) == "lower") sum(diff < 0) else sum(diff > 0)
    nz <- sum(diff != 0)
    if (nz > 0) stats::binom.test(fav, nz)$p.value else NA_real_
  },
  .groups = "drop"
) |>
dplyr::mutate(
  n_eff = n - n_ties,
  verdict = dplyr::case_when(
    n_eff == 0 ~ "no separation: all differences exactly tied",
    n_favour_diff == n_eff ~ "consistent: Diffusion",
    n_favour_diff == 0 ~ "consistent: GAN",
    TRUE ~ "inconsistent across phenotypes"),
  verdict = dplyr::if_else(n_ties > 0 & n_eff > 0,
    paste0(verdict, " (", n_ties, " tied)", verdict)) |>
dplyr::select(metric, better, n, n_ties, median_diff, mean_diff, sd_diff,
  n_favour_diff, p_paired_t, ci_lo, ci_hi, p_sign, verdict)

nice_kable(paired_stats, digits = 5,
  caption = paste("Pooled paired comparison, Diffusion minus",
    "GAN within each phenotype. The t-test is on the",
    "within-phenotype differences; the sign test excludes",
    "exact ties, so with five records it cannot fall below",
    "0.0625 and the unanimity count is the reportable",
    "result."))

```

Table 14: Pooled paired comparison, Diffusion minus GAN within each phenotype. The t-test is on the within-phenotype differences; the sign test excludes exact ties, so with five records it cannot fall below 0.0625 and the unanimity count is the reportable result.

metric	better	n	n_ties	median_diff	mean_diff	sd_diff	n_favour_diff	p_paired_t	
ACF RMS	lower	5	0	-0.01852	-0.03606	0.05175	5	0.19421	-0.0625
Envelope SD (pA)	lower	5	0	-0.12679	-0.28397	0.47175	3	0.24954	-0.0625
KS amplitude	lower	5	0	-0.03159	-0.00155	0.09698	3	0.97314	-0.0625
MMD envelope	lower	5	0	-0.01348	-0.02035	0.04230	4	0.34267	-0.0625
MMD permutation p	lower	5	1	0.00000	-0.11343	0.16950	2	0.20888	-0.0625
MMD vs real	lower	5	0	-0.00007	0.00195	0.00992	3	0.68357	-0.0625
MMD z-score	lower	5	0	0.20614	1.69927	14.65548	2	0.80823	-16.0625
Mean closed dwell (ms)	lower	5	0	-0.28285	-11.80649	21.40026	5	0.28487	-38.0625
Mean open dwell (ms)	lower	5	0	-4.51766	-17.06694	26.62782	4	0.22509	-50.0625
Noise SD (pA)	lower	5	0	-0.01315	-0.04349	0.07040	3	0.23931	-0.0625
Open probability	lower	5	0	-0.02330	-0.02114	0.04150	4	0.31830	-0.0625
log10 PSD RMS	lower	5	0	-0.03185	-0.12679	0.24361	3	0.30921	-0.0625

```
readr::write_csv(paired_stats, file.path(CFG$out_dir, "paired_summary.csv"))
```

What this many phenotypes will and will not support

```

min_p_sign <- min(1, 2 * 0.5^n_ph)
cat(sprintf("Phenotypes pooled: %d\n", n_ph))

```

```
## Phenotypes pooled: 5
cat(sprintf("Smallest two-sided p a sign test can return here: %.4f\n", min_p_sign))

## Smallest two-sided p a sign test can return here: 0.0625
if (min_p_sign > 0.05) {
  cat("\nThat is above 0.05, so no sign test on these data can reach significance,\n",
      "however large or however unanimous the effect. The same floor applies to the\n",
      "Wilcoxon signed-rank test at this n. The p_sign column above is therefore\n",
      "reported for completeness and should not be leaned on.\n",
      "\nWhat these data do support: the per-phenotype values, the direction, whether\n",
      "it is unanimous, and the paired t interval on the differences. Those are the\n",
      "defensible summaries at this sample size.\n", sep = "")
} else {
  cat("\nA sign test is at least capable of reaching p < 0.05 here.\n")
}

##
## That is above 0.05, so no sign test on these data can reach significance,
## however large or however unanimous the effect. The same floor applies to the
## Wilcoxon signed-rank test at this n. The p_sign column above is therefore
## reported for completeness and should not be leaned on.
##
## What these data do support: the per-phenotype values, the direction, whether
## it is unanimous, and the paired t interval on the differences. Those are the
## defensible summaries at this sample size.

cat(sprintf("\nUnanimous metrics: %d of %d.\n",
           sum(paired_stats$verdict != "inconsistent across phenotypes"),
           nrow(paired_stats)))

##
## Unanimous metrics: 3 of 12.
```

The paired t-test above assumes the five differences are roughly normal, which five points cannot demonstrate. It is quoted as an effect estimate with an interval rather than as a hypothesis test, and the interval is wide by construction. A metric that comes out inconsistent is a finding in itself: it means the two methods are close enough that recording-specific detail decides the ranking.

6. Publication figures

Figure: example traces

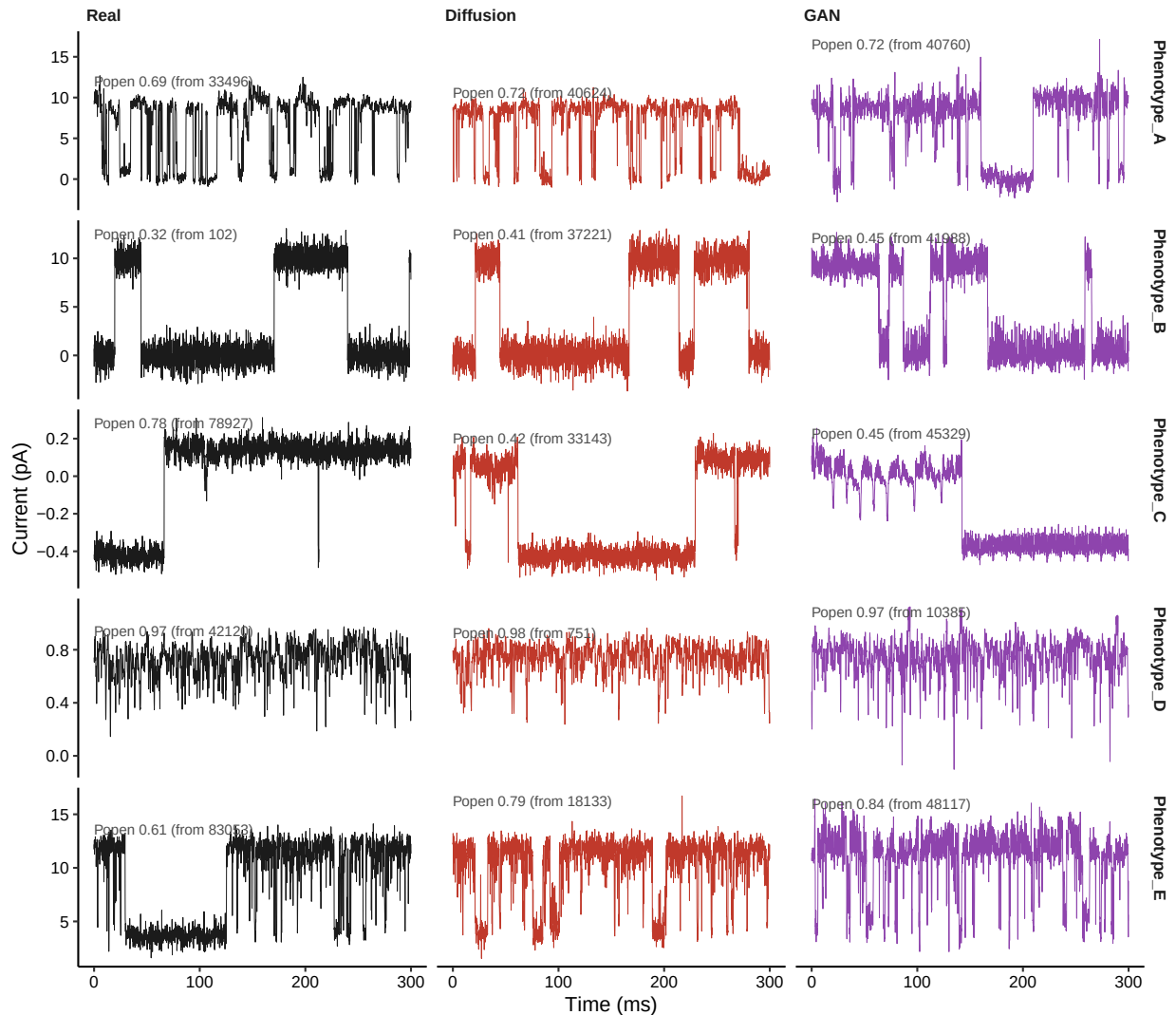
```
traces <- pull_all("traces") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")))

# Popen of the window actually shown, so a panel cannot quietly misrepresent the
# record it came from.
win_lab <- traces |>
  dplyr::group_by(phenotype, source) |>
  dplyr::summarise(win_popen = dplyr::first(win_popen),
                  win_start = dplyr::first(win_start),
                  x = min(time_ms), y = max(current), .groups = "drop") |>
  dplyr::mutate(label = sprintf("Popen %.2f (from %d)", win_popen, win_start))
```

```

p_traces <- ggplot(traces, aes(time_ms, current, colour = source)) +
  geom_line(linewidth = 0.2) +
  geom_text(data = win_lab, aes(x, y, label = label), hjust = 0, vjust = 1,
    size = 2.4, colour = "grey30", inherit.aes = FALSE) +
  facet_grid(phenotype ~ source, scales = "free_y") +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = "Time (ms)", y = "Current (pA)") +
  theme_channel()
p_traces

```



```

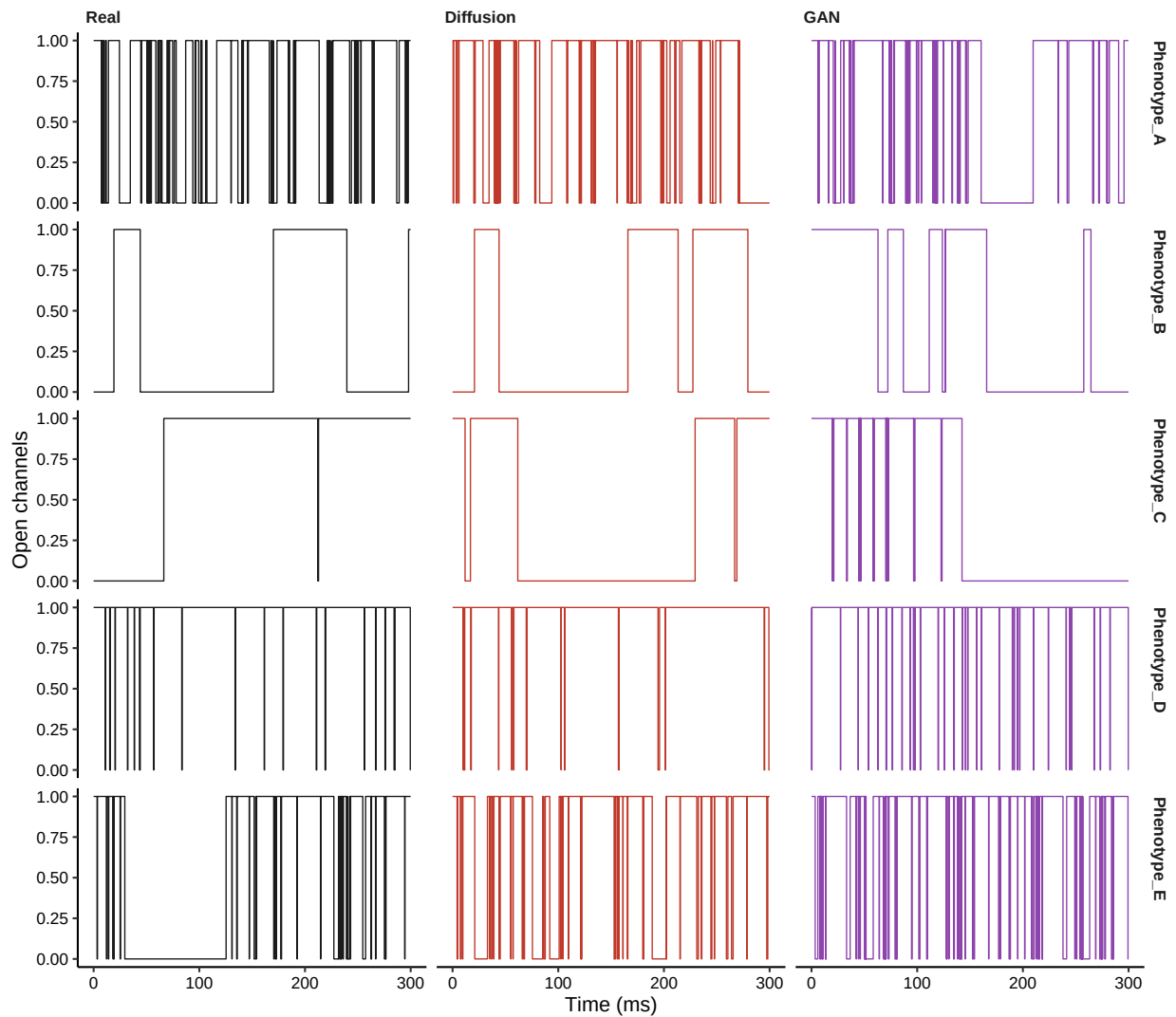
save_fig(p_traces, "fig_traces", width = 8, height = 1.4 * n_ph + 1)

```

The window is chosen by the same rule the Python previews use, so a panel shows gating rather than whichever stretch the file happens to start with. Where no window falls inside the target band — Phenotype_B is too quiet, Phenotype_D too busy — the nearest available is used, and its Popen is printed so the gap is visible rather than implied.

Figure: idealisations

```
p_ideal <- ggplot(traces, aes(time_ms, channels, colour = source)) +
  geom_step(linewidth = 0.3) +
  facet_grid(phenotype ~ source) +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = "Time (ms)", y = "Open channels") +
  theme_channel()
p_ideal
```



```
save_fig(p_ideal, "fig_idealisations", width = 8, height = 1.2 * n_ph + 1)
```

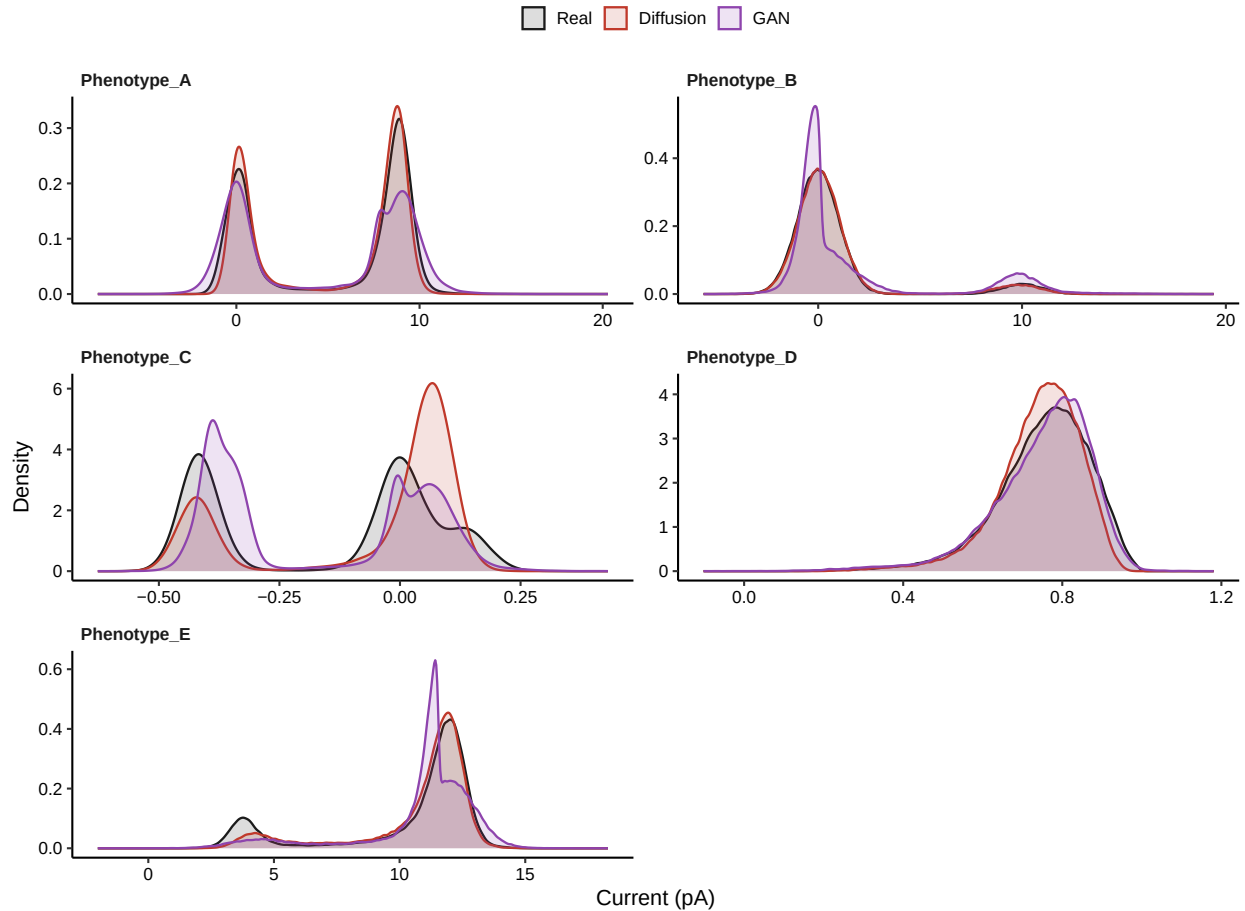
Figure: all-points amplitude histograms

```
p_amp <- pull_all("amp") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(current, colour = source, fill = source)) +
  geom_density(alpha = 0.15, linewidth = 0.5, adjust = 0.6) +
  facet_wrap(~ phenotype, scales = "free", ncol = 2) +
```

```

scale_colour_manual(values = pal) + scale_fill_manual(values = pal) +
labs(x = "Current (pA)", y = "Density") +
theme_channel()
p_amp

```



```

save_fig(p_amp, "fig_amplitude", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)

```

Figure: single-level noise

```

# Panels are labelled with the level they were computed at, so the legend can
# state it and a phenotype that offers no baseline is not silently different.
panel_label <- setNames(sprintf("%s (%s level, %d-sample segments)",
                                names(LEVEL_OF), LEVEL_OF,
                                NPERSEG_OF[names(LEVEL_OF)]),
                        names(LEVEL_OF))

p_psd <- pull_all("psd") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")),
                phenotype = panel_label[phenotype]) |>
  ggplot(aes(freq, psd, colour = source)) +
  geom_line(linewidth = 0.4) +
  geom_vline(xintercept = CFG$filter_hz, linetype = "dashed",
             colour = "grey60", linewidth = 0.3) +

```

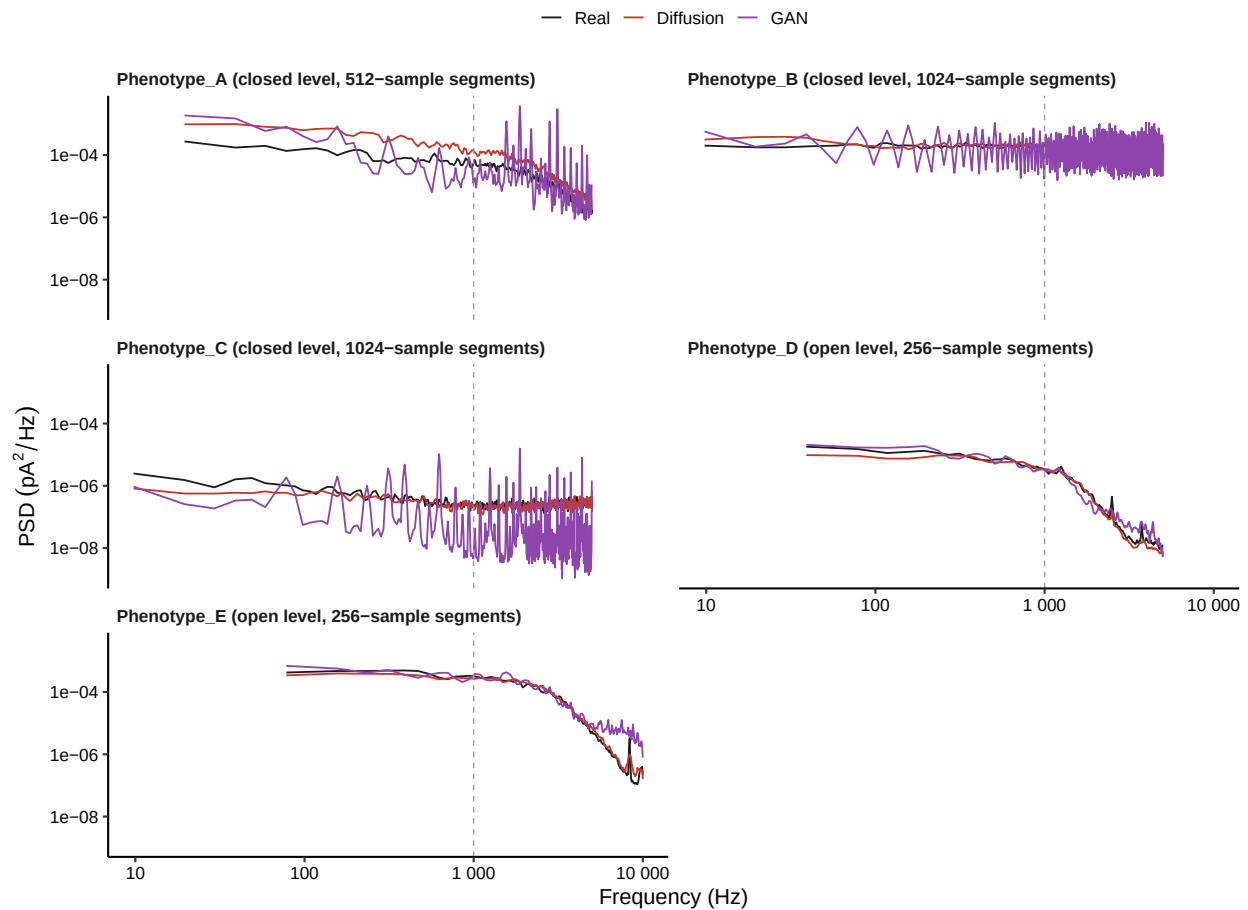
```

facet_wrap(~ phenotype, ncol = 2) +
scale_x_log10(labels = scales::label_number()) +
scale_y_log10(labels = scales::label_scientific()) +
scale_colour_manual(values = pal) +
labs(x = "Frequency (Hz)", y = expression(PSD~(pA^2/Hz))) +
theme_channel()

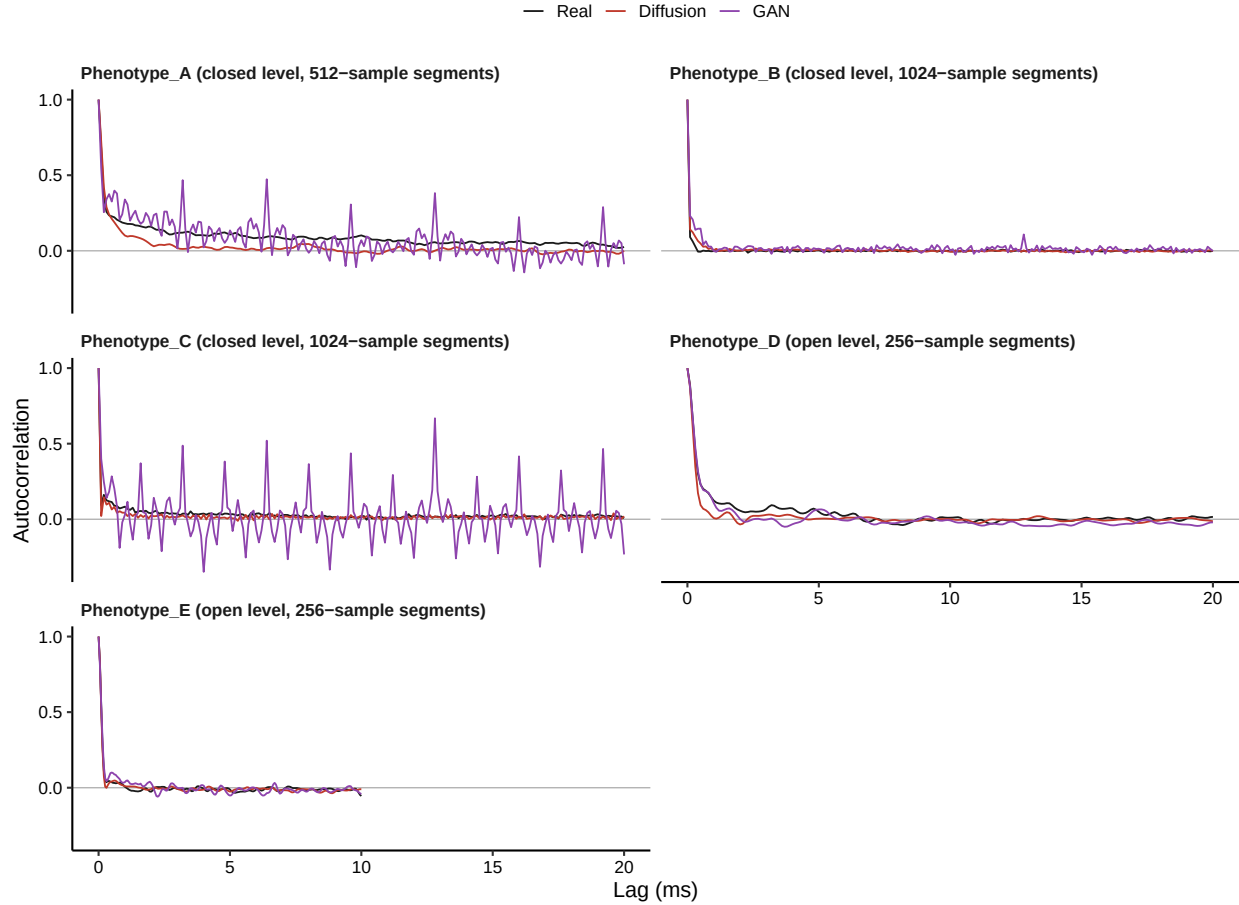
p_acf <- pull_all("acf") |>
dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")),
              phenotype = panel_label[phenotype]) |>
ggplot(aes(lag_ms, acf, colour = source)) +
geom_hline(yintercept = 0, colour = "grey70", linewidth = 0.3) +
geom_line(linewidth = 0.4) +
facet_wrap(~ phenotype, ncol = 2) +
scale_colour_manual(values = pal) +
labs(x = "Lag (ms)", y = "Autocorrelation") +
theme_channel()

p_psd

```



p_acf



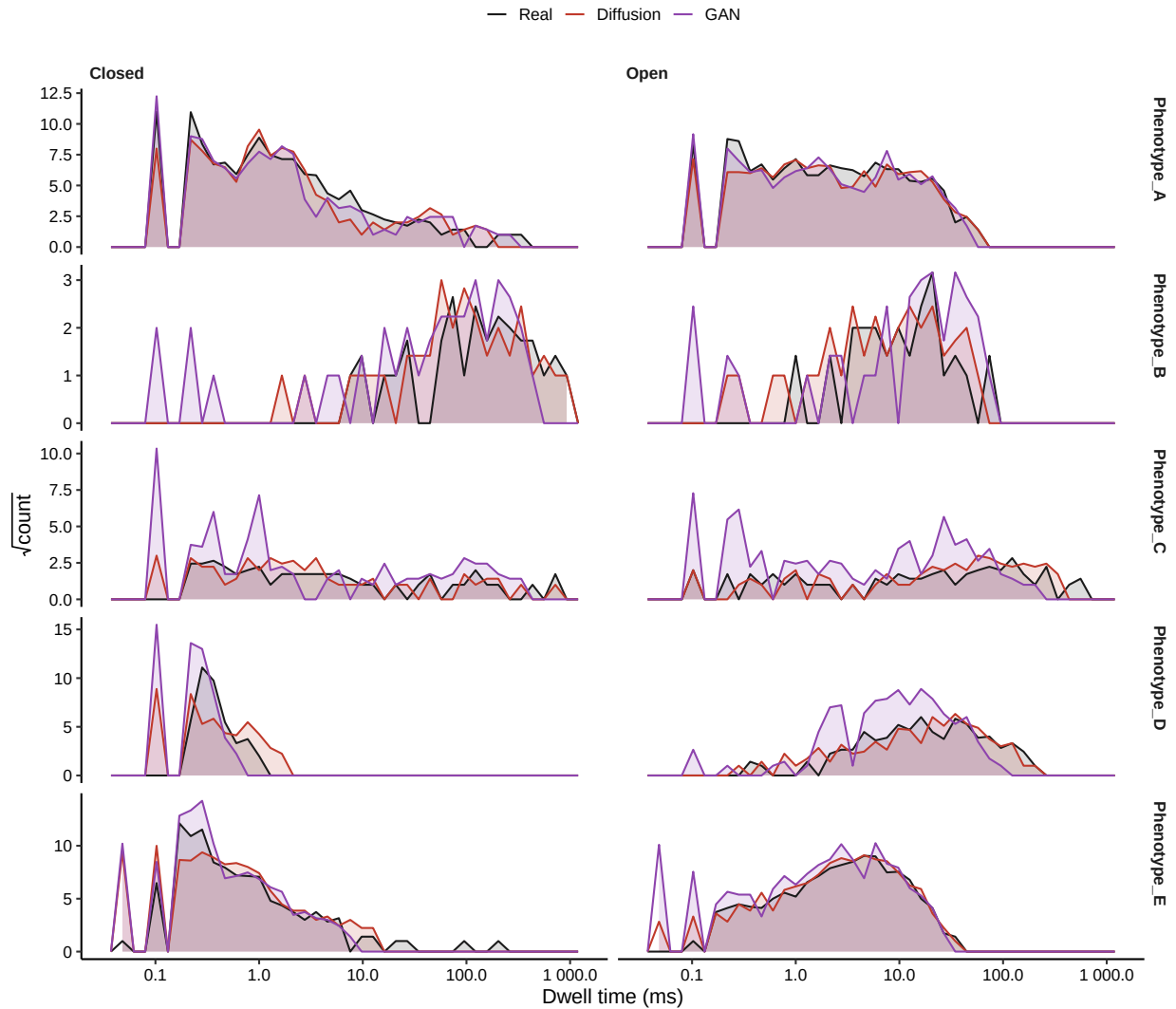
```
save_fig(p_psd, "fig_psd", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
save_fig(p_acf, "fig_acf", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
```

Figure: dwell times

Sigworth-Sine display: log abscissa, square-root ordinate. The diffusion arm's **Channels** column is the conditioning idealisation it was handed, so it is a known ground truth. The GAN emitted its own labels, so those are a network output. They are plotted together for comparison and must not be described as equivalent in the manuscript.

```
p_dwell <- pull_all("dwell") |>
  dplyr::filter(dwell_ms > 0) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(dwell_ms, colour = source)) +
  geom_area(aes(y = after_stat(sqrt(count)), fill = source),
    stat = "bin", bins = 40, alpha = 0.15,
    position = "identity", colour = NA) +
  geom_freqpoly(aes(y = after_stat(sqrt(count))), bins = 40, linewidth = 0.45) +
  facet_grid(phenotype ~ state, scales = "free_y") +
  scale_x_log10(labels = scales::label_number()) +
  scale_colour_manual(values = pal) +
  scale_fill_manual(values = pal, guide = "none") +
  labs(x = "Dwell time (ms)", y = expression(sqrt(count))) +
  theme_channel()
```

p_dwell



```
save_fig(p_dwell, "fig_dwell", width = 8, height = 1.5 * n_ph + 1)
```

and bars

```
d_dwell <- pull_all("dwell") |>
  dplyr::filter(dwell_ms > 0) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")))

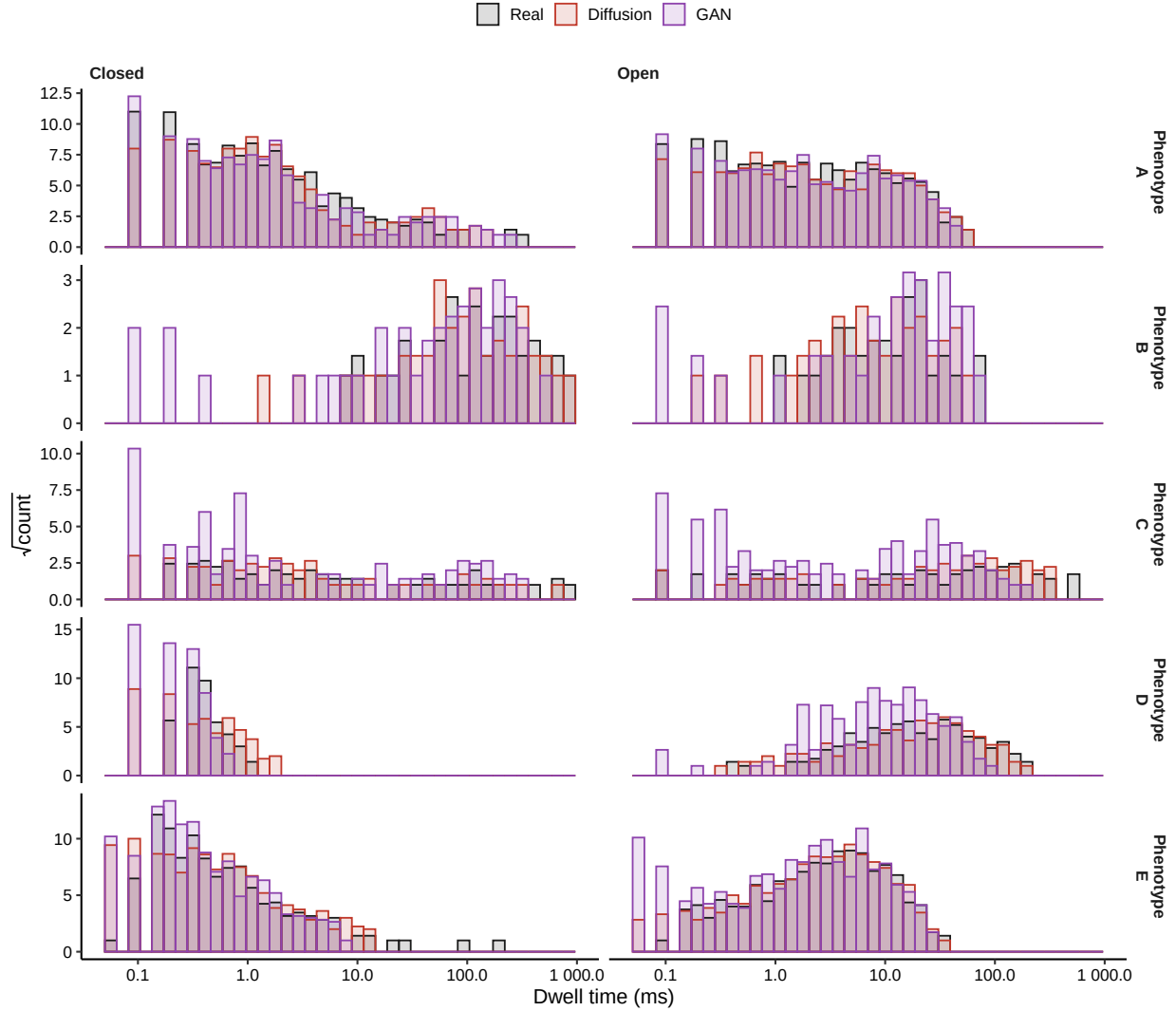
# Log-spaced breaks, so every bar is the same width on the log axis. Equal
# linear bins would be wide at short dwells and thin at long ones.
brk <- 10^seq(log10(min(d_dwell$dwell_ms)), log10(max(d_dwell$dwell_ms)),
  length.out = 41)

p_dwell <- ggplot(d_dwell, aes(dwell_ms, colour = source)) +
  geom_histogram(aes(y = after_stat(sqrt(count)), fill = source),
    breaks = brk, alpha = 0.15, position = "identity", colour = NA) +
  geom_histogram(aes(y = after_stat(sqrt(count))), fill = NA,
    breaks = brk, linewidth = 0.4, position = "identity") +
```

```

facet_grid(phenotype ~ state,
           scales = "free_y",
           labeller = labeller(
             phenotype = function(x) gsub("_", "\\n", x, fixed = TRUE)
           )) +
scale_x_log10(labels = scales::label_number()) +
scale_colour_manual(values = pal) +
scale_fill_manual(values = pal, guide = "none") +
labs(x = "Dwell time (ms)", y = expression(sqrt(count))) +
theme_channel()+
theme(
  strip.text.y = element_text(
    angle = 270,
    hjust = 0.5,
    vjust = 0.5
  )
) +
guides(colour = guide_legend(override.aes = list(
  fill = scales::alpha(pal[c("Real", "Diffusion", "GAN")], 0.15),
  linewidth = 0.4)))
p_dwell

```



```
save_fig(p_dwll, "fig_dwll", width = 8, height = 1.5 * n_ph + 1)
```

Bursts

*# Two-component exponential mixture on the closed times by EM, then the
Colquhoun-Sakmann equal-areas tcrit: the gap length at which as many brief
closures are misread as between-burst as long ones are misread as within.*

```
fit_tcrit <- function(closed_ms, n_iter = 200) {
  x <- closed_ms[is.finite(closed_ms) & closed_ms > 0]
  if (length(x) < 50) return(NA_real_)
  tau <- stats::quantile(x, c(0.25, 0.9), names = FALSE)
  if (!(tau[2] > tau[1])) return(NA_real_)
  a <- c(0.5, 0.5)
  for (i in seq_len(n_iter)) {
    d1 <- a[1] * stats::dexp(x, 1 / tau[1])
    d2 <- a[2] * stats::dexp(x, 1 / tau[2])
    w <- d1 / (d1 + d2); w[!is.finite(w)] <- 0.5
    a <- c(mean(w), 1 - mean(w))
  }
}
```

```

    tau <- c(sum(w * x) / sum(w), sum((1 - w) * x) / sum(1 - w))
    if (any(!is.finite(tau)) || any(tau <= 0)) return(NA_real_)
  }
  o <- order(tau); tau <- tau[o]; a <- a[o]
  # A record with one closed component has no bursts to find, and forcing a
  # tcrit onto it invents them.
  if (tau[2] / tau[1] < 3 || min(a) < 0.02) return(NA_real_)
  f <- function(t) a[1] * exp(-t / tau[1]) - a[2] * (1 - exp(-t / tau[2]))
  lo <- tau[1] * 1e-3; hi <- tau[2]
  if (!is.finite(f(lo)) || !is.finite(f(hi)) || f(lo) * f(hi) > 0) return(NA_real_)
  stats::uniroot(f, c(lo, hi), tol = 1e-9)$root
}

burst_of <- function(dw, tcrit) {
  if (!is.finite(tcrit)) return(NULL)
  sep <- dw$state == "Closed" & dw$dwell_ms >= tcrit
  b <- cumsum(sep)
  d <- dw[!sep, , drop = FALSE]; b <- b[!sep]
  if (!nrow(d)) return(NULL)
  # The first and last bursts are cut off by the ends of the record, so both
  # would drag the mean burst length down.
  inner <- b > min(b) & b < max(b)
  d <- d[inner, , drop = FALSE]; b <- b[inner]
  if (!nrow(d)) return(NULL)
  tibble::tibble(burst = b, state = d$state, ms = d$dwell_ms) |>
    dplyr::group_by(burst) |>
    dplyr::summarise(length_ms = sum(ms),
                     n_open    = sum(state == "Open"),
                     open_ms   = sum(ms[state == "Open"]),
                     n_gap     = sum(state == "Closed"),
                     gap_ms    = sum(ms[state == "Closed"]),
                     .groups = "drop") |>
    dplyr::filter(n_open > 0)
}

res_burst <- purrr::map(manifest$phenotype, function(ph) {
  arms <- recs[[ph]]; fs <- FS_MAIN[[ph]]
  dwl <- lapply(names(arms), function(a) dwell_times(arms[[a]]$channels, fs[[a]]))
  names(dwl) <- names(arms)
  # One tcrit per phenotype, taken from the real record and imposed on every
  # arm. Per-arm tcrits would make the burst counts incomparable.
  tcrit <- fit_tcrit(dwl$Real$dwell_ms[dwl$Real$state == "Closed"])
  per_burst <- purrr::map_dfr(names(arms), function(a) {
    b <- burst_of(dwl[[a]], tcrit)
    if (is.null(b) || !nrow(b)) return(NULL)
    dplyr::mutate(b, phenotype = ph, source = a)
  })
  if (!nrow(per_burst)) return(list(summary = NULL, per_burst = NULL))
  list(
    per_burst = per_burst,
    summary = per_burst |>
      dplyr::group_by(phenotype, source) |>
      dplyr::summarise(

```

```

    tcrit_ms = tcrit, n_bursts = dplyr::n(),
    `Burst length (ms)` = mean(length_ms),
    `Openings per burst` = mean(n_open),
    `Open time in burst (ms)` = sum(open_ms) / sum(n_open),
    `Gap in burst (ms)` = if (sum(n_gap)) sum(gap_ms) / sum(n_gap) else NA_real_,
    `Burst Popen` = sum(open_ms) / sum(length_ms),
    .groups = "drop") |>
dplyr::mutate(`Bursts per s` = purrr::map_dbl(source, function(a)
  sum(per_burst$source == a) / (nrow(arms[[a]]) / fs[[a]])))
})

burst <- purrr::map_dfr(res_burst, "summary")
bursts_raw <- purrr::map_dfr(res_burst, "per_burst")

nice_kable(dplyr::select(burst, phenotype, source, tcrit_ms, n_bursts,
  dplyr::everything()), digits = 4,
  caption = paste("Burst kinetics. tcrit is fitted on the",
    "real record of each phenotype and applied to all",
    "three arms; a phenotype whose closed times are",
    "monoexponential yields no tcrit and drops out."))

```

Table 15: Burst kinetics. tcrit is fitted on the real record of each phenotype and applied to all three arms; a phenotype whose closed times are monoexponential yields no tcrit and drops out.

phenotype	source	tcrit_ms	n_bursts	Burst length (ms)	Openings per burst	Open time in burst (ms)
Phenotype_A	Diffusion	4.2582	62	78.9032	12.4194	5.4114
Phenotype_A	GAN	4.2582	79	54.1696	9.7595	4.8442
Phenotype_A	Real	4.2582	88	56.6080	10.1250	4.7424
Phenotype_C	Diffusion	7.8043	15	484.2067	6.0667	78.6407
Phenotype_C	GAN	7.8043	47	97.1149	5.9574	15.8129
Phenotype_C	Real	7.8043	17	356.0647	4.0588	86.4014
Phenotype_E	Diffusion	2.0843	71	64.7042	12.3803	4.8287
Phenotype_E	GAN	2.0843	42	113.7774	27.1429	3.8190
Phenotype_E	Real	2.0843	55	74.7109	14.2000	4.8590

```

burst_cmp <- burst |>
  dplyr::select(-tcrit_ms, -n_bursts) |>
  tidyr::pivot_longer(-c(phenotype, source), names_to = "metric") |>
  tidyr::pivot_wider(names_from = source, values_from = value) |>
  dplyr::filter(is.finite(Real), is.finite(Diffusion), is.finite(GAN)) |>
  dplyr::mutate(err_diffusion = abs(Diffusion - Real),
    err_gan = abs(GAN - Real),
    favours = dplyr::case_when(
      err_diffusion < err_gan ~ "Diffusion",
      err_gan < err_diffusion ~ "GAN",
      TRUE ~ "Tie"
    ))

nice_kable(burst_cmp, digits = 4,
  caption = "Burst metrics, absolute error against the real record.")

```

Table 16: Burst metrics, absolute error against the real record.

phenotype	metric	Diffusion	GAN	Real	err_diffusion	err_gan	favours
Phenotype_A	Burst length (ms)	78.9032	54.1696	56.6080	22.2953	2.4383	GAN
Phenotype_A	Openings per burst	12.4194	9.7595	10.1250	2.2944	0.3655	GAN
Phenotype_A	Open time in burst (ms)	5.4114	4.8442	4.7424	0.6690	0.1018	GAN
Phenotype_A	Gap in burst (ms)	1.0243	0.7868	0.9415	0.0828	0.1546	Diffusion
Phenotype_A	Burst Popen	0.8518	0.8728	0.8482	0.0035	0.0245	Diffusion
Phenotype_A	Bursts per s	8.4565	10.7753	12.0028	3.5463	1.2276	GAN
Phenotype_C	Burst length (ms)	484.2067	97.1149	356.0647	128.1420	258.9498	Diffusion
Phenotype_C	Openings per burst	6.0667	5.9574	4.0588	2.0078	1.8986	GAN
Phenotype_C	Open time in burst (ms)	78.6407	15.8129	86.4014	7.7608	70.5886	Diffusion
Phenotype_C	Gap in burst (ms)	1.4053	0.5871	1.7577	0.3524	1.1706	Diffusion
Phenotype_C	Burst Popen	0.9853	0.9700	0.9849	0.0004	0.0149	Diffusion
Phenotype_C	Bursts per s	1.5000	4.7000	1.7000	0.2000	3.0000	Diffusion
Phenotype_E	Burst length (ms)	64.7042	113.7774	74.7109	10.0067	39.0665	Diffusion
Phenotype_E	Openings per burst	12.3803	27.1429	14.2000	1.8197	12.9429	Diffusion
Phenotype_E	Open time in burst (ms)	4.8287	3.8190	4.8590	0.0304	1.0400	Diffusion
Phenotype_E	Gap in burst (ms)	0.4327	0.3871	0.4328	0.0001	0.0457	Diffusion
Phenotype_E	Burst Popen	0.9239	0.9111	0.9235	0.0004	0.0125	Diffusion
Phenotype_E	Bursts per s	14.2000	8.4000	11.0000	3.2000	2.6000	GAN

```
nice_kable(
  burst_cmp |>
    dplyr::group_by(metric) |>
    dplyr::summarise(n = dplyr::n(),
                     n_favour_diffusion = sum(favours == "Diffusion"),
                     .groups = "drop"),
  caption = "Unanimity of the burst metrics across phenotypes.")
```

Table 17: Unanimity of the burst metrics across phenotypes.

metric	n	n_favour_diffusion
Burst Popen	3	3
Burst length (ms)	3	2
Bursts per s	3	1
Gap in burst (ms)	3	3
Open time in burst (ms)	3	2
Openings per burst	3	1

```
readr::write_csv(burst, file.path(CFG$out_dir, "burst_kinetics.csv"))
readr::write_csv(burst_cmp, file.path(CFG$out_dir, "burst_comparison.csv"))
```

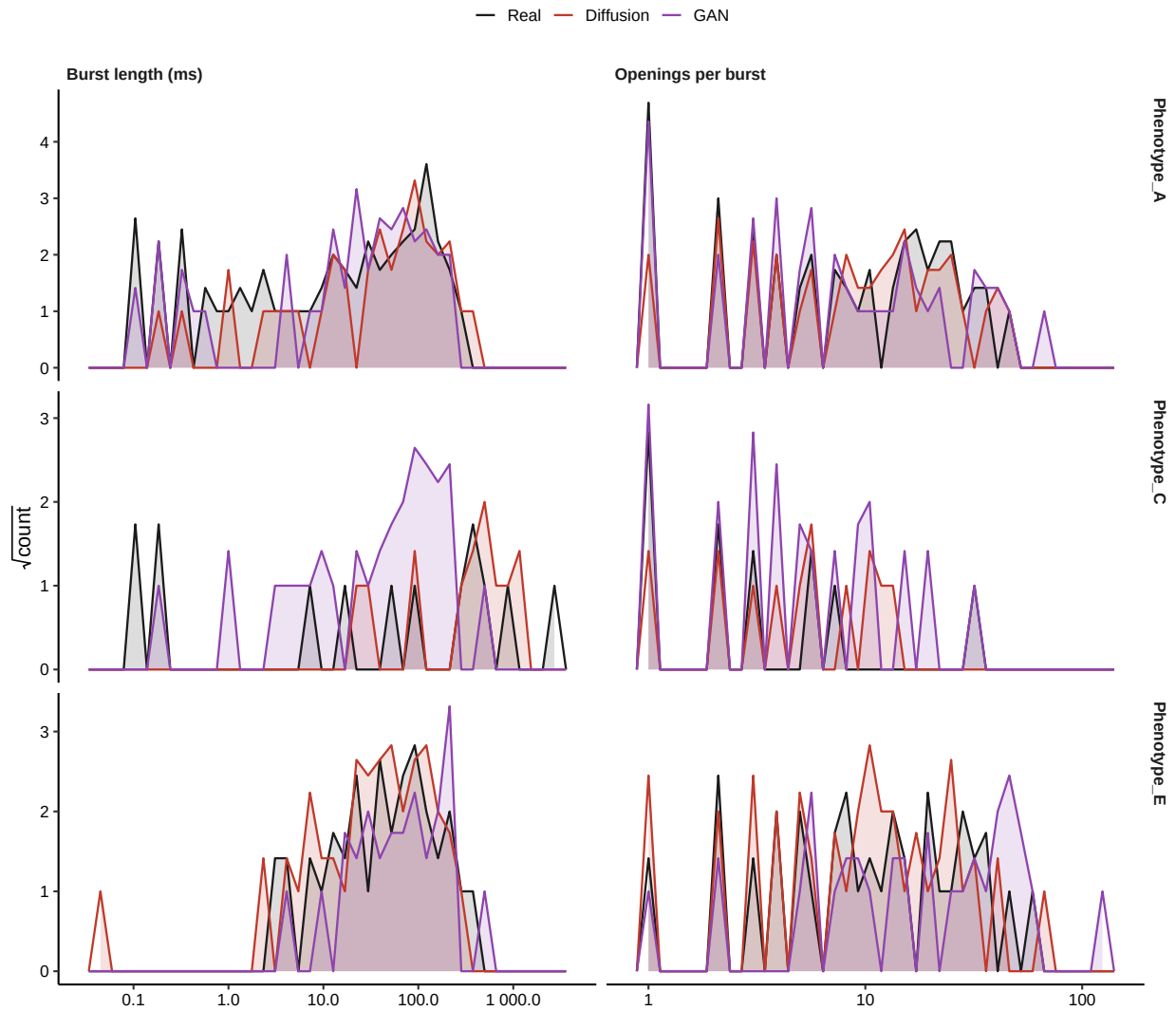
Burst Figures

```
p_burst <- bursts_raw |>
  dplyr::transmute(phenotype, source,
    `Burst length (ms)` = length_ms,
    `Openings per burst` = as.numeric(n_open)) |>
```

```

tidyr::pivot_longer(-c(phenotype, source), names_to = "quantity") |>
dplyr::filter(value > 0) |>
dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
ggplot(aes(value, colour = source)) +
geom_area(aes(y = after_stat(sqrt(count)), fill = source),
          stat = "bin", bins = 40, alpha = 0.15,
          position = "identity", colour = NA) +
geom_freqpoly(aes(y = after_stat(sqrt(count))), bins = 40, linewidth = 0.5) +
facet_grid(phenotype ~ quantity, scales = "free") +
scale_x_log10(labels = scales::label_number()) +
scale_colour_manual(values = pal) +
scale_fill_manual(values = pal, guide = "none") +
labs(x = NULL, y = expression(sqrt(count))) +
theme_channel()
p_burst

```



```

save_fig(p_burst, "fig_burst", width = 8, height = 1.5 * n_ph + 1)
readr::write_csv(bursts_raw, file.path(CFG$out_dir, "bursts_per_burst.csv"))

```


bars

```
d_burst <- bursts_raw |>
  dplyr::transmute(phenotype, source,
    `Burst length (ms)` = length_ms,
    `Openings per burst` = as.numeric(n_open)) |>
  tidyr::pivot_longer(-c(phenotype, source), names_to = "quantity") |>
  dplyr::filter(value > 0) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")))

# Breaks per quantity, since burst length and opening count span quite
# different ranges, and log-spaced so every bar is one width on the log axis.
log_breaks <- function(v, n = 41) {
  r <- range(v)
  if (r[1] == r[2]) return(NULL)
  10^seq(log10(r[1]), log10(r[2]), length.out = n)
}

brk_burst <- split(d_burst$value, d_burst$quantity) |> lapply(log_breaks)

layer_hist <- function(q) {
  d <- dplyr::filter(d_burst, quantity == q)
  list(
    geom_histogram(data = d, aes(y = after_stat(sqrt(count))), fill = source),
    breaks = brk_burst[[q]], alpha = 0.15,
    position = "identity", colour = NA),
    geom_histogram(data = d, aes(y = after_stat(sqrt(count))), fill = NA,
    breaks = brk_burst[[q]], linewidth = 0.4,
    position = "identity")
  )
}

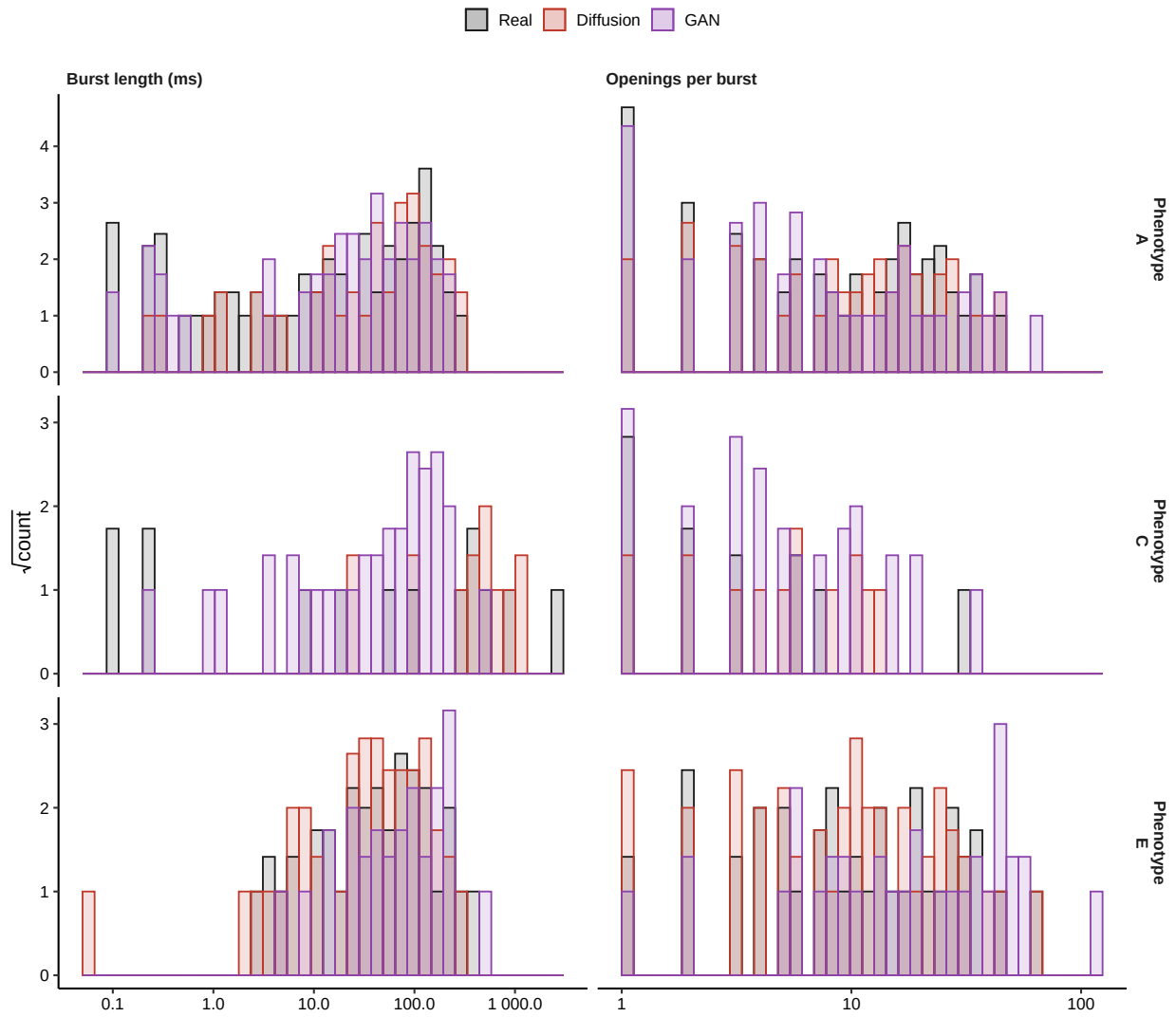
p_burst <- ggplot(d_burst, aes(value, colour = source)) +
  lapply(names(brk_burst), layer_hist) +
  facet_grid(
    phenotype ~ quantity,
    scales = "free",
    labeller = labeller(
      phenotype = function(x) gsub("_", "\\n", x, fixed = TRUE)
    )
  ) +
  scale_x_log10(labels = scales::label_number()) +
  scale_colour_manual(values = pal) +
  scale_fill_manual(values = pal, guide = "none") +
  labs(
    x = NULL,
    y = expression(sqrt(count))
  ) +
  theme_channel() +
  theme(
    strip.text.y = element_text(
      angle = 270,
      hjust = 0.5,
      vjust = 0.5
    )
  )
```

```

)
) +
guides(
  colour = guide_legend(
    override.aes = list(
      fill = scales::alpha(
        pal[c("Real", "Diffusion", "GAN")],
        0.15
      ),
    ),
    linewidth = 0.4
  )
)
)
)

```

p_burst



```

save_fig(p_burst, "fig_burst", width = 8, height = 1.5 * n_ph + 1)
readr::write_csv(bursts_raw, file.path(CFG$out_dir, "bursts_per_burst.csv"))

```

Label self-consistency

Carried over from `DiffuChannel_analysis.Rmd`, where it ran on one record at a time. Each arm's current trace is idealised on its own, with a conventional half-amplitude criterion and no reference to any other arm, and the result is compared against the `Channels` column that arm shipped with.

This asks a different question from everything above. The GAN emitted its labels and its trace together, so there is no external truth to score them against – but there is an internal one: the labels should agree with what the arm's own current is doing. The diffusion arm was handed its labels before it started, so its agreement measures whether the generated current followed the conditioning it was given. The real record supplies the floor, because expert idealisation and a half-amplitude criterion do not agree perfectly either, and an arm cannot reasonably be asked to do better than the seed.

```
res_selfcon <- purrr::map(manifest$phenotype, function(ph) {

  arms <- recs[[ph]]
  fs   <- FS_MAIN[[ph]]

  # First pass: try unsupervised EM on every arm.
  em_rec <- purrr::map(names(arms), function(a) {

    d <- arms[[a]]

    n_lev <- max(
      1,
      max(d$channels, na.rm = TRUE)
    )

    idealise_half_amplitude(
      d$current,
      n_levels = n_lev,
      polarity = POL_MAIN[[ph]]
    )
  })

  names(em_rec) <- names(arms)

  # If EM fails for any arm, use label-anchored CV for every
# arm in this phenotype, keeping the comparison consistent.
  em_failed <- vapply(
    em_rec,
    function(x) all(is.na(x)),
    logical(1)
  )

  phenotype_fallback <- any(em_failed)

  if (phenotype_fallback) {
    failed_arms <- names(arms)[em_failed]

    method_note <- paste0(
      "Phenotype-wide fallback because unsupervised EM failed for: ",
      paste(failed_arms, collapse = ", ")
    )
  } else {
```

```

method_note <- ""
}

# Second pass: calculate self-consistency using the method
# selected for this entire phenotype.
out <- purrr::map(names(arms), function(a) {

  d <- arms[[a]]

  n_lev <- max(
    1,
    max(d$channels, na.rm = TRUE)
  )

  lab <- d$channels > 0

  if (phenotype_fallback) {

    rec <- idealise_label_anchored_cv(
      current = d$current,
      labels = d$channels,
      n_levels = n_lev,
      polarity = POL_MAIN[[ph]],
      block_len = 1000L,
      trim = 0.10,
      min_state_n = 50L
    )

    ideal_method <- "Phenotype-wide label-anchored CV fallback"

  } else {

    rec <- em_rec[[a]]
    ideal_method <- "Unsupervised EM"
  }

  # This occurs only if the selected phenotype-wide
  # method still fails for this arm.
  if (all(is.na(rec))) {

    failure_reason <- attr(rec, "why")

    if (is.null(failure_reason)) {
      failure_reason <- "no reason was recorded"
    }

  }

  return(list(
    summary = tibble::tibble(
      phenotype = ph,
      source = a,
      kappa = NA_real_,
      fi_open = NA_real_,
      popen_labels = mean(lab),

```

```

    popen_recovered = NA_real_,
    open_ms_labels = NA_real_,
    open_ms_recovered = NA_real_,
    flicker = NA,
    ideal_method = ideal_method,
    note = paste(
      method_note,
      "Selected method failed:",
      failure_reason
    )
  ),
  dwell = NULL
))
}

op <- rec > 0

tp <- sum(lab & op)
fp <- sum(!lab & op)
fn <- sum(lab & !op)

dl <- dwell_times(
  as.integer(lab),
  fs[[a]]
)

dr <- dwell_times(
  as.integer(op),
  fs[[a]]
)

mo <- function(x) {
  v <- x$dwell_ms[x$state == "Open"]

  if (length(v)) {
    mean(v)
  } else {
    NA_real_
  }
}

list(
  summary = tibble::tibble(
    phenotype = ph,
    source = a,
    kappa = cohen_kappa(lab, op),
    fl_open = if (2 * tp + fp + fn > 0) {
      2 * tp / (2 * tp + fp + fn)
    } else {
      NA_real_
    },
    popen_labels = mean(lab),

```

```

    popen_recovered = mean(op),
    open_ms_labels = mo(dl),
    open_ms_recovered = mo(dr),
    flicker = length(rle(op)$lengths) > 0.5 * length(op),
    ideal_method = ideal_method,
    note = method_note
  ),

  dwell = dplyr::bind_rows(
    dplyr::mutate(
      dl,
      origin = "Own labels"
    ),
    dplyr::mutate(
      dr,
      origin = "Recovered from current"
    )
  ) |>
  dplyr::mutate(
    phenotype = ph,
    source = a
  )
)
})

list(
  summary = purrr::map_dfr(out, "summary"),
  dwell = purrr::map_dfr(out, "dwell")
)
})

self_consist <- purrr::map_dfr(res_selfcon, "summary")
sc_dwell <- purrr::map_dfr(res_selfcon, "dwell")

if (any(self_consist$flicker %in% TRUE)) {
  warning("The recovered idealisation switches on more than half the samples ",
    "in: ", paste(sprintf("%s/%s", self_consist$phenotype[self_consist$flicker %in% TRUE],
      self_consist$source[self_consist$flicker %in% TRUE]),
      collapse = ", "),
    ". That is a wrong level estimate rather than a fast record, and the ",
    "kappa for those rows means nothing. Check the amplitude histogram.")
}

# A recovered Popen that is the complement of the labelled one is an inverted
# idealisation, not a disagreement, and its kappa is the negative of the real
# number. It should not happen now the polarity is imposed, so say so loudly.
inverted <- self_consist |>
  dplyr::filter(is.finite(kappa),
    abs(popen_recovered - (1 - popen_labels)) < 0.05,
    kappa < 0)
if (nrow(inverted)) {
  warning("The idealisation looks inverted in: ",
    paste(sprintf("%s/%s", inverted$phenotype, inverted$source),

```

```

        collapse = ", "), ". Check POL_MAIN for those phenotypes.")
}

nice_kable(self_consist, digits = 4,
  caption = paste("Label self-consistency. Each arm's",
    "Channels column against a half-amplitude",
    "idealisation of that same arm's current. The real",
    "record is the floor, not a target of 1.))")

```

Table 18: Label self-consistency
against a half-amplitude idealisation
The real record is the floor, not a target of 1.

phenotype	source	kappa	fl_open	popen_labels	popen_recovered	open_ms_labels	open_ms_recovered
Phenotype_A	Real	0.9500	0.9792	0.5825	0.5871	4.7505	4.9500
Phenotype_A	Diffusion	0.9538	0.9806	0.5784	0.5826	5.4371	5.0900
Phenotype_A	GAN	0.9519	0.9783	0.5461	0.5506	4.9070	6.1000
Phenotype_B	Real	0.9723	0.9743	0.0715	0.0715	15.5522	15.5500
Phenotype_B	Diffusion	0.9206	0.9267	0.0790	0.0751	14.1071	9.8800
Phenotype_B	GAN	0.9620	0.9680	0.1592	0.1592	21.5149	10.1400
Phenotype_C	Real	0.9935	0.9974	0.6037	0.6006	79.4382	25.3400
Phenotype_C	Diffusion	0.9789	0.9948	0.7548	0.7469	78.6260	11.3500
Phenotype_C	GAN	0.9810	0.9903	0.4858	0.4915	15.6190	21.7400
Phenotype_D	Real	0.3454	0.9783	0.9883	0.9462	31.8800	5.5600
Phenotype_D	Diffusion	0.3689	0.9797	0.9879	0.9486	32.0744	5.6600
Phenotype_D	GAN	0.3298	0.9726	0.9849	0.9345	14.3362	4.5800
Phenotype_E	Real	0.9434	0.9898	0.8171	0.8226	4.8987	4.2500
Phenotype_E	Diffusion	0.9149	0.9880	0.8571	0.8598	4.8150	3.7900
Phenotype_E	GAN	0.8529	0.9827	0.8804	0.8850	3.8244	4.6200

```

sc_cmp <- self_consist |>
  dplyr::select(phenotype, source, kappa) |>
  tidyr::pivot_wider(names_from = source, values_from = kappa) |>
  dplyr::filter(
    is.finite(Real),
    is.finite(Diffusion),
    is.finite(GAN)
  ) |>
  dplyr::mutate(
    deficit_diffusion = Real - Diffusion,
    deficit_gan       = Real - GAN,

    error_diffusion = abs(Diffusion - Real),
    error_gan       = abs(GAN - Real),

    pct_error_diffusion = 100 * error_diffusion / abs(Real),
    pct_error_gan       = 100 * error_gan / abs(Real),

    favours = dplyr::case_when(
      error_diffusion < error_gan ~ "Diffusion",
      error_gan < error_diffusion ~ "GAN",

```

```

    TRUE ~ "Tie"
  )
)

nice_kable(sc_cmp, digits = 4,
  caption = paste("Shortfall in kappa against the real",
    "record's own floor. A negative deficit means the",
    "arm's labels agree with its current better than",
    "the expert idealisation agrees with the seed,",
    "which is a sign the arm is too clean rather than",
    "a sign it is good.))

```

Table 19: Shortfall in kappa against the real record's own floor. A negative deficit means the arm's labels agree with its current better than the expert idealisation agrees with the seed, which is a sign the arm is too clean rather than a sign it is good.

phenotype	Real	Diffusion	GAN	deficit_diffusion	deficit_gan	error_diffusion	error_gan	pct_error_c
Phenotype_A	0.9500	0.9538	0.9519	-0.0038	-0.0020	0.0038	0.0020	
Phenotype_B	0.9723	0.9206	0.9620	0.0517	0.0103	0.0517	0.0103	
Phenotype_C	0.9935	0.9789	0.9810	0.0145	0.0125	0.0145	0.0125	
Phenotype_D	0.3454	0.3689	0.3298	-0.0235	0.0156	0.0235	0.0156	
Phenotype_E	0.9434	0.9149	0.8529	0.0285	0.0905	0.0285	0.0905	

```

cat(sprintf("\nPhenotypes where the diffusion arm is the more self-consistent: %d of %d.\n",
  sum(sc_cmp$favours == "Diffusion"), nrow(sc_cmp)))

```

```

##
## Phenotypes where the diffusion arm is the more self-consistent: 1 of 5.

```

```

readr::write_csv(self_consist, file.path(CFG$out_dir, "self_consistency.csv"))
readr::write_csv(sc_cmp, file.path(CFG$out_dir, "self_consistency_comparison.csv"))

```

The gap between the two dwell distributions below is the missed-event loss: the brief openings and closures that a half-amplitude criterion cannot recover from the current, however good the trace is. Being able to measure it at all is what having the conditioning idealisation buys.

```

pal_origin <- c("Own labels" = "#1b1b1b",
  "Recovered from current" = "#c0392b")

d_sc <- sc_dwell |>
  dplyr::filter(dwell_ms > 0) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")),
    origin = factor(origin, levels = names(pal_origin)))

brk_sc <- 10^seq(log10(min(d_sc$dwell_ms)), log10(max(d_sc$dwell_ms)),
  length.out = 41)

p_selfcon <- ggplot(d_sc, aes(dwell_ms, colour = origin)) +
  geom_histogram(aes(y = after_stat(sqrt(count)), fill = origin),
    breaks = brk_sc, alpha = 0.15, position = "identity",
    colour = NA) +
  geom_histogram(aes(y = after_stat(sqrt(count))), fill = NA,
    breaks = brk_sc, linewidth = 0.4, position = "identity") +

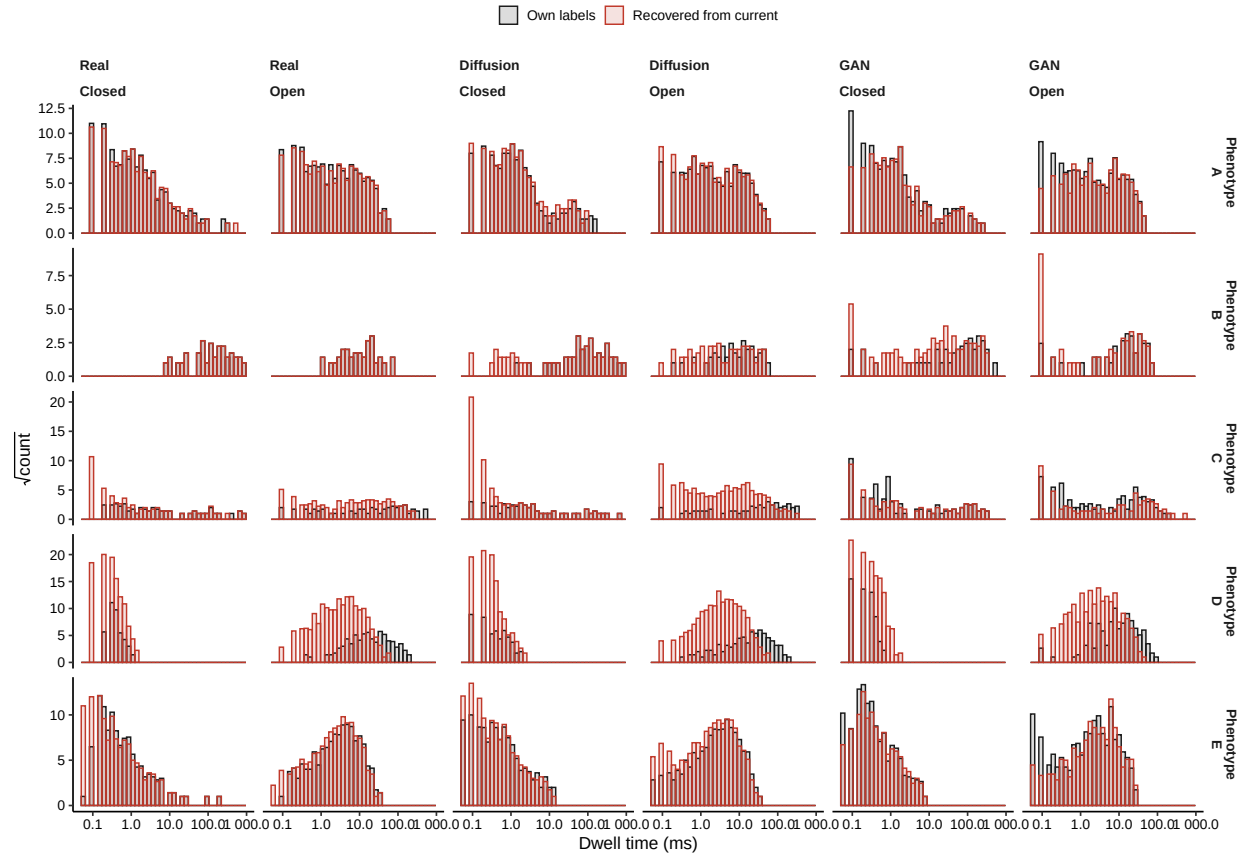
```



```

facet_grid(
  phenotype ~ source + state,
  scales = "free_y",
  labeller = labeller(
    phenotype = function(x) gsub("_", "\\n", x, fixed = TRUE)
  )
) +
scale_x_log10(labels = scales::label_number()) +
scale_colour_manual(values = pal_origin) +
scale_fill_manual(values = pal_origin, guide = "none") +
labs(x = "Dwell time (ms)", y = expression(sqrt(count))) +
theme_channel() +
theme(
  strip.text.y = element_text(
    angle = 270,
    hjust = 0.5,
    vjust = 0.5
  )
) +
guides(
  colour = guide_legend(
    override.aes = list(
      fill = scales::alpha(pal_origin, 0.15),
      linewidth = 0.4
    )
  )
)
p_selfcon

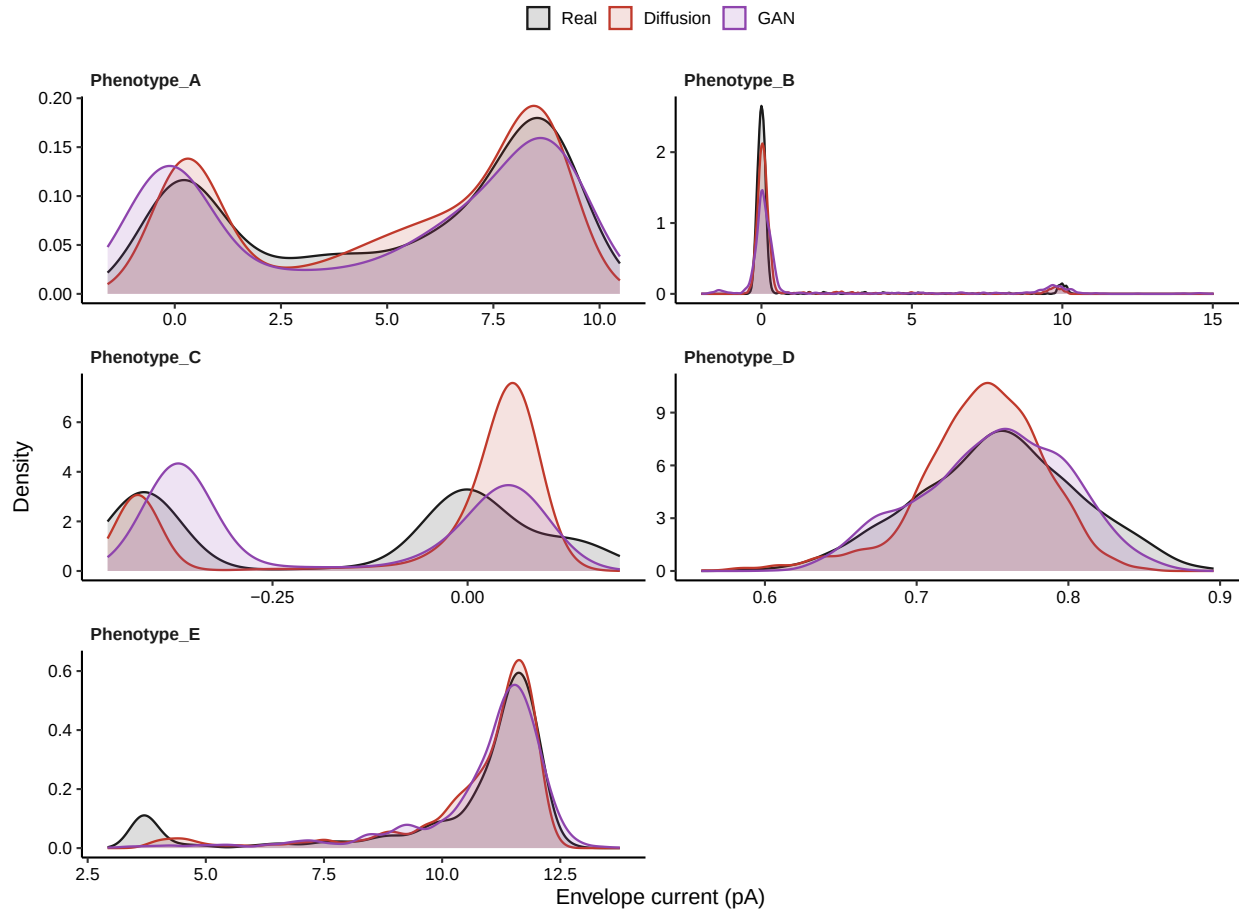
```



```
save_fig(p_selfcon, "fig_self_consistency", width = 10, height = 1.5 * n_ph + 1)
readr::write_csv(sc_dwell, file.path(CFG$out_dir, "self_consistency_dwells.csv"))
```

Figure: slow envelope

```
p_env <- pull_all("env") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(value, colour = source, fill = source)) +
  geom_density(alpha = 0.15, linewidth = 0.5) +
  facet_wrap(~ phenotype, scales = "free", ncol = 2) +
  scale_colour_manual(values = pal) + scale_fill_manual(values = pal) +
  labs(x = "Envelope current (pA)", y = "Density") +
  theme_channel()
p_env
```



```
save_fig(p_env, "fig_envelope", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
```

Figure: manifold projections (optional)

```
if (!all(vapply(c("uwot", "Rtsne"), requireNamespace, logical(1), quietly = TRUE))) {
  stop("Set CFG$do_manifold = FALSE or install uwot and Rtsne.")
}

proj <- purrr::map_dfr(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  d <- list(Real = read_record(row$raw), Diffusion = read_record(row$ddpm))
  if (!is.na(row$gan)) d$GAN <- read_record(row$gan)
  Wp <- lapply(d, function(x) make_windows(x$current, CFG$proj_win, CFG$n_windows_proj))
  X <- do.call(rbind, Wp)
  lab <- rep(names(Wp), vapply(Wp, nrow, integer(1)))
  X <- scale(X); X[!is.finite(X)] <- 0
  um <- uwot::umap(X, n_neighbors = 15, min_dist = 0.1, n_components = 2)
  ts <- Rtsne::Rtsne(X, dims = 2, perplexity = 30, check_duplicates = FALSE,
    pca = TRUE)$Y
  dplyr::bind_rows(
    tibble::tibble(x = um[, 1], y = um[, 2], source = lab, method = "UMAP"),
    tibble::tibble(x = ts[, 1], y = ts[, 2], source = lab, method = "t-SNE")
  ) |> dplyr::mutate(phenotype = row$phenotype)
```

```

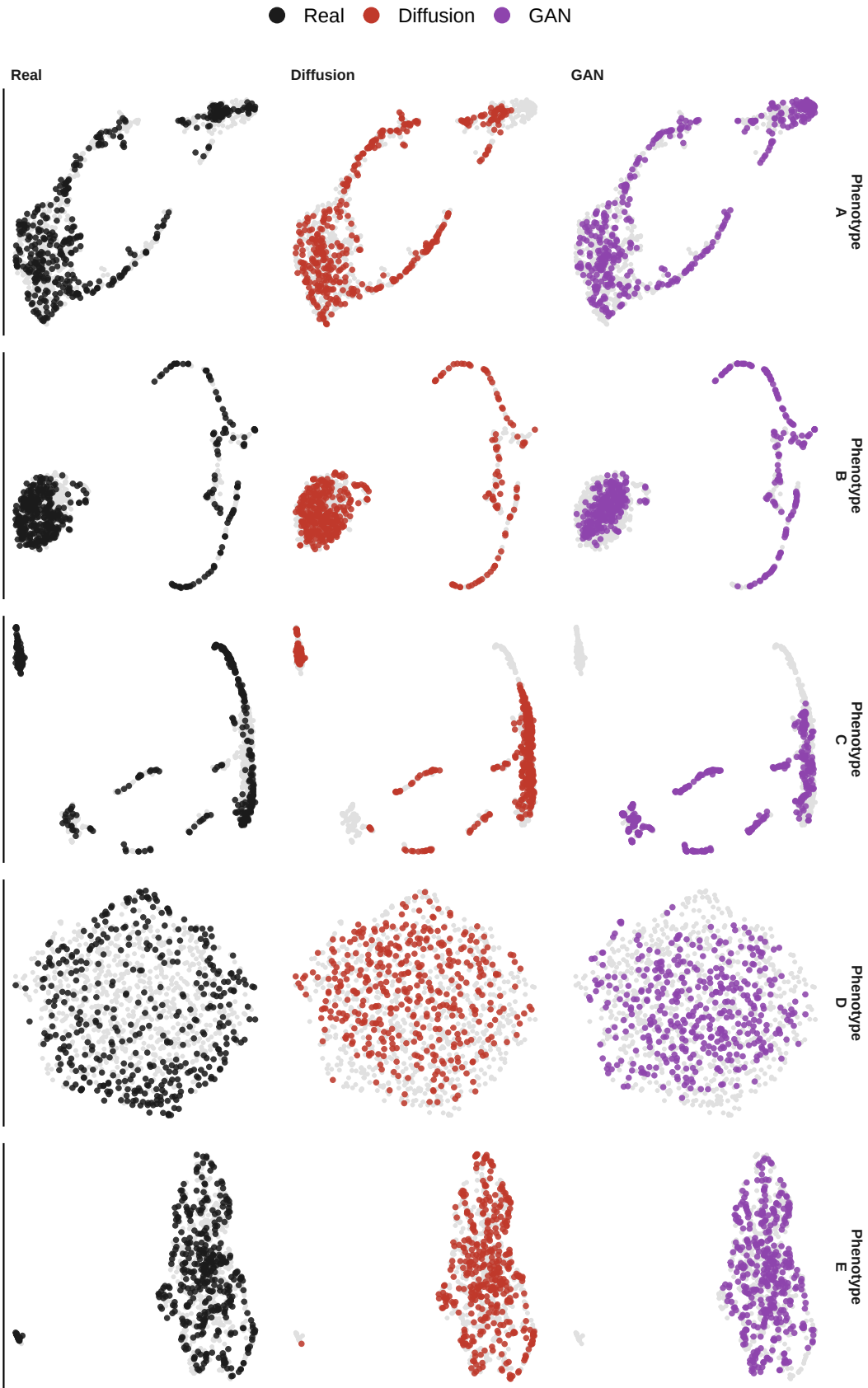
})

proj <- proj |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  dplyr::group_by(phenotype, method) |>
  dplyr::mutate(x = scales::rescale(x), y = scales::rescale(y)) |>
  dplyr::ungroup()

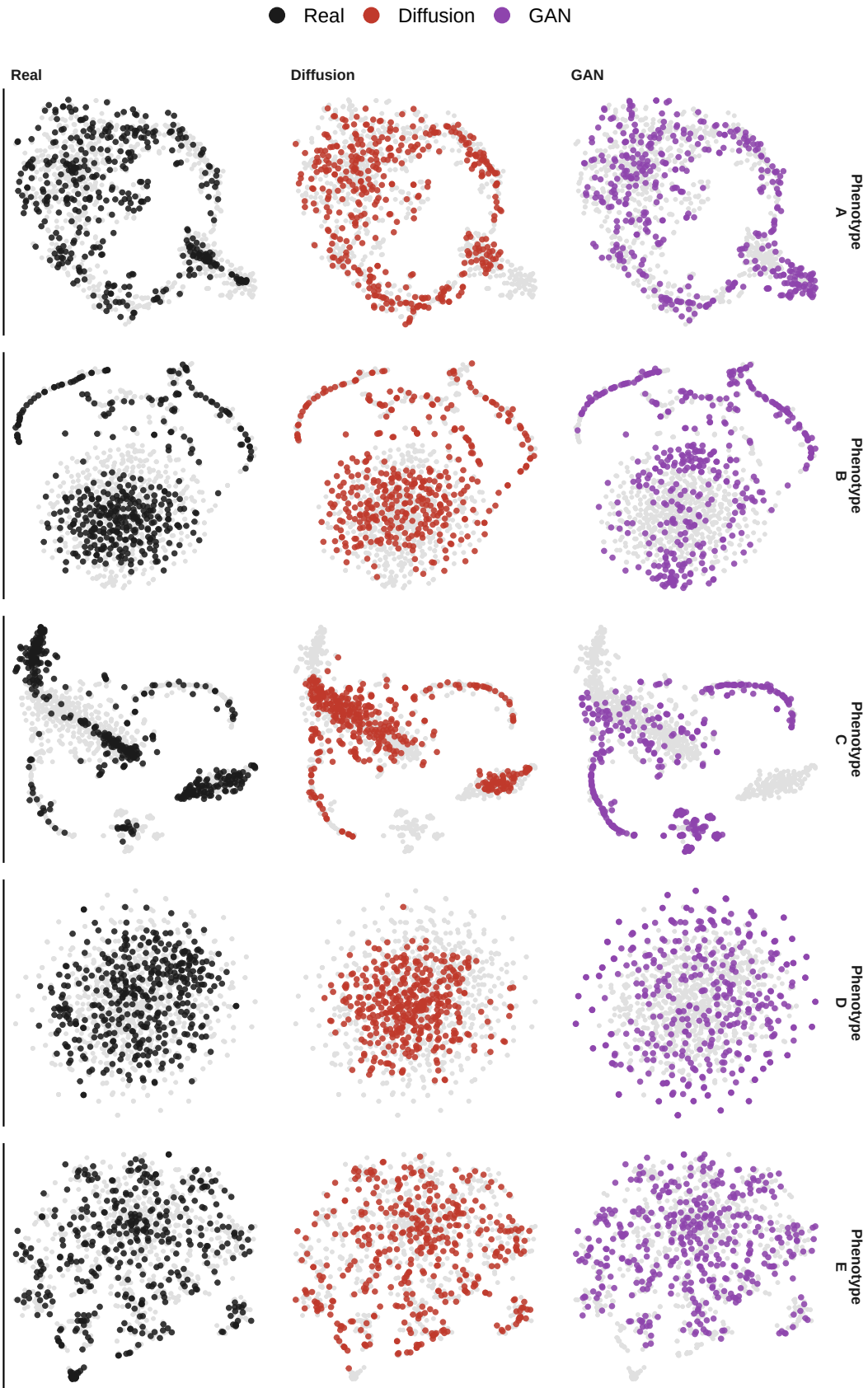
# One arm per column, over a grey ghost of the full cloud so each panel still
# shows where the other two sat. Rescaling per phenotype lets the three columns
# share fixed scales, which free scales previously prevented.
plot_proj <- function(d) {
  ggplot(d, aes(x, y)) +
    geom_point(
      data = dplyr::select(d, -source),
      colour = "grey88",
      size = 0.7
    ) +
    geom_point(
      aes(colour = source),
      size = 1.1,
      alpha = 0.85
    ) +
    facet_grid(
      phenotype ~ source,
      labeller = labeller(
        phenotype = function(x) gsub("_", "\\n", x)
      )
    ) +
    scale_colour_manual(values = pal) +
    guides(
      colour = guide_legend(
        override.aes = list(size = 4, alpha = 1)
      )
    ) +
    labs(x = NULL, y = NULL) +
    theme_channel(base_size = 11) +
    theme(
      axis.text = element_blank(),
      axis.ticks = element_blank(),
      legend.text = element_text(size = 12),
      legend.key.size = unit(1.5, "lines"),
      panel.spacing = unit(0.7, "lines"),
      strip.text.y = element_text(
        angle = 270,
        hjust = 0.5,
        vjust = 0.5
      )
    )
}

p_proj_umap <- plot_proj(dplyr::filter(proj, method == "UMAP"))
p_proj_tsne <- plot_proj(dplyr::filter(proj, method == "t-SNE"))
p_proj_umap

```



p_proj_tsne



```
save_fig(p_proj_umap, "fig_manifold_umap", width = 7.5, height = 2.3 * n_ph + 1)
save_fig(p_proj_tsne, "fig_manifold_tsne", width = 7.5, height = 2.3 * n_ph + 1)
readr::write_csv(proj, file.path(CFG$out_dir, "manifold_coordinates.csv"))
```

7. Numbers for the manuscript

Rows keyed to `[[placeholder]]` names. Pooled values are medians across phenotypes, with the range, because five records do not support a mean and an interval being read as an estimate.

```
fmt <- function(x, d = 4) formatC(x, digits = d, format = "g")

pooled <- scored |>
  dplyr::group_by(metric, method) |>
  dplyr::summarise(median = median(value), lo = min(value), hi = max(value),
    .groups = "drop")

placeholders <- dplyr::bind_rows(
  tibble::tibble(placeholder = "n_phenotypes", value = as.character(n_ph),
    note = "paired seed records"),
  tibble::tibble(placeholder = "n_samples_total",
    value = format(sum(info$n_real), big.mark = ","),
    note = "seed samples across all phenotypes"),
  # Rate before unique, or floating-point dust splits one rate into several.
  tibble::tibble(placeholder = "sample_khz",
    value = paste(unique(round(sort(info$fs_real) / 1000, 3)),
      collapse = ", "),
    note = "acquisition rate(s), kHz"),
  pooled |>
  dplyr::transmute(
    placeholder = paste0(gsub("[^A-Za-z0-9]+", "_", tolower(metric)), "_",
      tolower(method)),
    value = sprintf("%s (%s to %s)", fmt(median), fmt(lo), fmt(hi)),
    note = paste("median across phenotypes, range in brackets")),
  paired_stats |>
  dplyr::transmute(
    placeholder = paste0("unanimity_", gsub("[^A-Za-z0-9]+", "_", tolower(metric))),
    value = sprintf("%d/%d favour Diffusion", n_favour_diff, n),
    note = verdict)
)

nice_kable(placeholders, caption = "Values keyed to the manuscript placeholders.")
```

Table 20: Values keyed to the manuscript placeholders.

placeholder	value	note
n_phenotypes	5	paired seed records
n_samples_total	473,316	seed samples across all phenotypes
sample_khz	10, 20	acquisition rate(s), kHz
acf_rms_diffusion	0.01909 (0.01165 to 0.06887)	median across phenotypes, range in brackets
acf_rms_gan	0.03908 (0.0263 to 0.1464)	median across phenotypes, range in brackets
envelope_sd_pa_diffusion	1.728 (0.04133 to 3.476)	median across phenotypes, range in brackets
envelope_sd_pa_gan	1.535 (0.04825 to 3.928)	median across phenotypes, range in brackets

ks_amplitude_diffusion	0.07343 (0.01977 to 0.2849)	median across phenotypes, range in brackets
ks_amplitude_gan	0.123 (0.02207 to 0.1478)	median across phenotypes, range in brackets
mmd_envelope_diffusion	0.009693 (2.885e-05 to 0.04906)	median across phenotypes, range in brackets
mmd_envelope_gan	0.02272 (0.007959 to 0.08477)	median across phenotypes, range in brackets
mmd_permutation_p_diffusion	0.004975 (0.004975 to 0.4229)	median across phenotypes, range in brackets
mmd_permutation_p_gan	0.00995 (0.004975 to 0.05473)	median across phenotypes, range in brackets
mmd_vs_real_diffusion	0.004778 (-0.0003896 to 0.06394)	median across phenotypes, range in brackets
mmd_vs_real_gan	0.004848 (0.001776 to 0.04871)	median across phenotypes, range in brackets
mmd_z_score_diffusion	5.543 (-0.224 to 28.82)	median across phenotypes, range in brackets
mmd_z_score_gan	5.336 (1.793 to 20.36)	median across phenotypes, range in brackets
mean_closed_dwell_ms_diffusion	3.957 (0.3932 to 161.6)	median across phenotypes, range in brackets
mean_closed_dwell_ms_gan	4.083 (0.2201 to 112.1)	median across phenotypes, range in brackets
mean_open_dwell_ms_diffusion	14.11 (4.815 to 78.63)	median across phenotypes, range in brackets
mean_open_dwell_ms_gan	14.34 (3.824 to 21.51)	median across phenotypes, range in brackets
noise_sd_pa_diffusion	0.999 (0.04001 to 1.333)	median across phenotypes, range in brackets
noise_sd_pa_gan	1.164 (0.05438 to 1.519)	median across phenotypes, range in brackets
open_probability_diffusion	0.7548 (0.079 to 0.9879)	median across phenotypes, range in brackets
open_probability_gan	0.5461 (0.1592 to 0.9849)	median across phenotypes, range in brackets
log10_psd_rms_diffusion	0.1781 (0.09052 to 0.6356)	median across phenotypes, range in brackets
log10_psd_rms_gan	0.3331 (0.105 to 0.8146)	median across phenotypes, range in brackets
unanimity_acf_rms	5/5 favour Diffusion	consistent: Diffusion
unanimity_envelope_sd_pa	3/5 favour Diffusion	inconsistent across phenotypes
unanimity_ks_amplitude	3/5 favour Diffusion	inconsistent across phenotypes
unanimity_mmd_envelope	4/5 favour Diffusion	inconsistent across phenotypes
unanimity_mmd_permutation_p	2/5 favour Diffusion	inconsistent across phenotypes (1 tied)
unanimity_mmd_vs_real	3/5 favour Diffusion	inconsistent across phenotypes
unanimity_mmd_z_score	2/5 favour Diffusion	inconsistent across phenotypes
unanimity_mean_closed_dwell_ms	5/5 favour Diffusion	consistent: Diffusion
unanimity_mean_open_dwell_ms	4/5 favour Diffusion	inconsistent across phenotypes
unanimity_noise_sd_pa	3/5 favour Diffusion	inconsistent across phenotypes
unanimity_open_probability	4/5 favour Diffusion	inconsistent across phenotypes
unanimity_log10_psd_rms	3/5 favour Diffusion	inconsistent across phenotypes

What this comparison does and does not settle

Both methods are scored identically above, so the numbers are comparable as measures of how much the synthetic data resemble the real record. That is the only question they answer.

They do not settle the question the paper is about. The GAN emits its raw trace and its **Channels** column together, so its labels are a network output; the diffusion pipeline is handed its labels before it starts and never touches them. A GAN record could match the real data better on every metric here and still be the less useful training set, because a downstream idealiser trained against it is being scored on another network's opinion rather than on truth. If the GAN wins on similarity, say so plainly and make the ground-truth argument on its own terms.

Session information

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
```

```

## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] Rtsne_0.17      uwot_0.2.3      Matrix_1.7-3    patchwork_1.3.1
## [5] arrow_21.0.0.1  purrr_1.1.0     readr_2.1.5     ggplot2_3.5.2
## [9] tidyr_1.3.1     dplyr_1.2.1
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4  xml2_1.4.0      stringi_1.8.7   lattice_0.22-6
## [5] hms_1.1.3       digest_0.6.37   magrittr_2.0.3  evaluate_1.0.4
## [9] grid_4.5.0      RColorBrewer_1.1-3 fastmap_1.2.0   viridisLite_0.4.2
## [13] scales_1.4.0    textshaping_1.0.1 cli_3.6.5       crayon_1.5.3
## [17] rlang_1.3.0     bit64_4.6.0-1   withr_3.0.2     yaml_2.3.10
## [21] FNN_1.1.4.1     parallel_4.5.0  tools_4.5.0     tzdb_0.5.0
## [25] kableExtra_1.4.0 assertthat_0.2.1 vctrs_0.7.3     R6_2.6.1
## [29] lifecycle_1.0.5 stringr_1.5.1    bit_4.6.0       vroom_1.6.5
## [33] irlba_2.3.5.1   ragg_1.4.0      pkgconfig_2.0.3 pillar_1.11.0
## [37] gtable_0.3.6    glue_1.8.0      Rcpp_1.1.0      systemfonts_1.3.2
## [41] xfun_0.53       tibble_3.3.0    tidyselect_1.2.1 rstudioapi_0.17.1
## [45] knitr_1.50      farver_2.1.2    htmltools_0.5.8.1 labeling_0.4.3
## [49] rmarkdown_2.29  svglite_2.2.2   compiler_4.5.0

```